CasePath: Agents That Read, Rules That Decide

Requesting Evidence for Insurance Claims with Executable Workflows

NAVISH KUMAR, University of Basel, Switzerland
TOBIAS HOESLI, ETH Zurich, Switzerland
AYDIN JAVADOV, ETH Zurich, Switzerland
JOSEPH OLLIER, ETH Zurich, Switzerland

Case handlers must request evidence before a file settles the deciding facts. In CasePath, a model reads each such fact several times and must quote the file. Code admits a value only when the readings agree, executes the organisation's rules, and holds every request that depends on an open fact. We tested a frozen commit on sealed synthetic claims under a pre-registered plan, against designs that give the same model the same rules. On tenancy claims with an open or denied deciding fact it made 0.34 unsafe errors per claim; a single prompt applying the rules made 2.32 on a cost-efficient model and 2.13 on a frontier model. Where every fact was stated it had no advantage over the frontier prompt (0.22 against 0.19). On the cost-efficient model its mean error was below that of every single-prompt and prior-work design on tenancy and on theft; three of five registered hypotheses were confirmed, and its theft lead over self-consistency missed significance. More samples of the same prompt did not close the gap. On the frontier model, run once, it led on tenancy and had no advantage on theft, where the rules alone account for 0.333 of its 0.833 errors per pair. Two blind AI evaluators reproduced the tenancy references exactly. The lead holds under alternative readings of the rules when the workflow follows them, and reverses under one when it does not. The design holds more than needed and costs several times a single prompt. Code, synthetic data and analysis are released.

CCS Concepts: • **Applied computing → Business process management**; *Law*; • **Computing methodologies → Knowledge representation and reasoning**; *Natural language processing*.

Additional Key Words and Phrases: LLM agents, evidence requests, claims handling, rule execution, three-valued logic, supervaluation, neuro-symbolic systems, pre-registered evaluation, design science

1 INTRODUCTION

A single sentence in a claim can decide which documents a case handler should request. A tenant reports mould, and the insurer's rules require a medical confirmation if the mould has affected someone's health. One version of the claim says "the paediatrician suspects a link with the mould"; another says "we do not know whether it has anything to do with the mould". For the first, the handler should request the confirmation. For the second, the handler should hold it and ask one question (Figure 1). A request made too early costs the claimant and the handler time, a document dropped too early weakens the decision, and an insurer must be able to show why it asked.

This is one instance of a class of problems: deciding which evidence to request while the file leaves deciding facts open. We treat it as design-science research [20, 22]. We build an artefact for the class, state design principles with their boundary conditions, and evaluate the artefact in one confirmatory test.

The idea is a division of labour. A language model reads each deciding fact several times and must quote the file for every value. Code admits a value only when the readings agree and the quotation is found, executes the organisation's rules, and holds every request whose need depends on a fact the file leaves open. A single prompt that applies the rules gives one answer per document; it records no fact values that code could check, and it does not keep a fact the file omits apart from one it denies or leaves open.

Authors' addresses: Navish Kumar, University of Basel, Basel, Switzerland, navish.kumar@unibas.ch; Tobias Hoesli, ETH Zurich, Zurich, Switzerland; Aydin Javadov, ETH Zurich, Zurich, Switzerland; Joseph Ollier, ETH Zurich, Zurich, Switzerland.

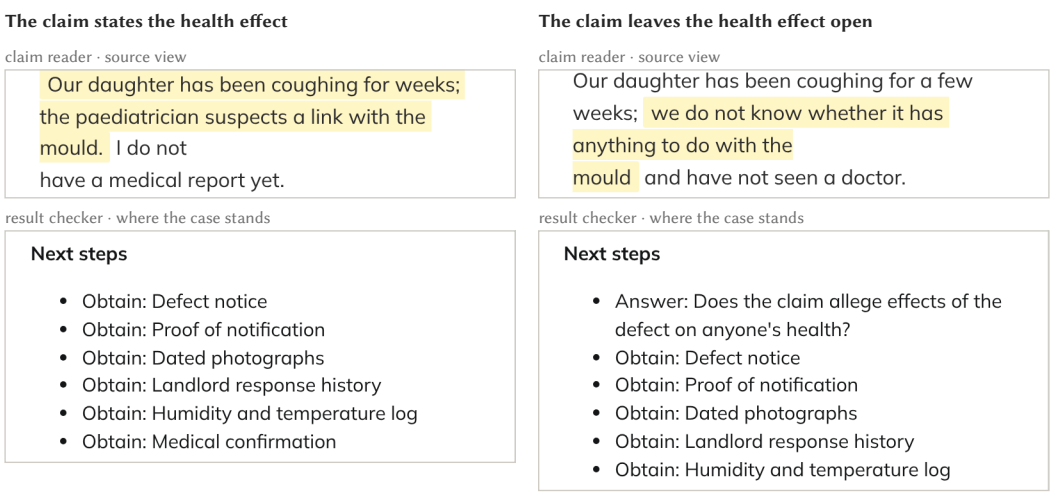


Fig. 1. The running example in CasePath: one changed sentence in the claim turns the request for the medical confirmation into a question, and the other five requests are unchanged. Top: the claim reader's source view, with the quoted words highlighted. Bottom: "Next steps", written by the result checker, a model agent, from the checklist that code evaluated. The lease is attached to both claims, so it is present and not requested; Figure 2 shows the workflow without files. Unedited captures of the product at commit 6a72159c on two synthetic claims run with the tenancy defect workflow; cropped.

We implemented the idea in CasePath, a product that prepares legal-protection claims for a case handler, and tested it at a frozen commit on sealed synthetic claims under a pre-registered plan (Section 6). Every comparator received the same files and the same rules. The main finding is that the design *holds* what the file leaves open. On the 56 of 96 tenancy claims with an open fact or a denied or open defining event, CasePath made 0.34 unsafe errors per claim; an unsafe error is a request the rules do not allow or a needed document left out. The single prompt with the rules made 2.32 on the cost-efficient model and 2.13 on the frontier model. On the 40 claims that state every fact, CasePath made 0.22 and the frontier prompt 0.19: no advantage. This split was added after review. In the registered results, CasePath left 94% of theft pairs with an unresolved fact otherwise unchanged, against 27–47% for every other design given the reviewed rules. It held 97% of the tenancy documents the reference holds, against 48% and 35% for the single prompts.

Our claim is a same-model claim. On the cost-efficient model, CasePath had a lower mean error than every single-prompt and prior-work design in both data sets. The registered tests confirmed this against the single prompt in both data sets and against two of three prior-work strategies on theft. Against self-consistency, a majority vote over several samples, the theft lead (0.58 against 1.21 errors per pair) gave $p = 0.0625$, and that hypothesis failed. On the frontier model, run once, CasePath made fewer tenancy errors and had no advantage on theft: 0.83 errors per pair against 0.95 for the frontier prompt with the same rules and 0.67 with the original rules. Of five registered hypotheses, three were confirmed, one failed and one was not tested; one confirmation depends on a reordering of the hypotheses shortly before the freeze.

The frontier theft cell shows where the design stops helping. With every fact read correctly, the reviewed theft rules still cause 0.333 errors per pair; most of the rest comes from two over-broad rule conditions read literally (Section 6.7). The theft rules had been reviewed by AI editor sessions that knew the test conditions; with the original rules, the single prompt made 1.47 errors per pair

on the cost-efficient model and 0.67 on the frontier model, the best single prompt on theft. A lint that flags such conditions without reference answers found them, but a remedy built on its flags did not survive out of sample. The design also holds more than needed and costs several times a single prompt.

The article makes three contributions:

(1) A decision rule that admits only agreeing, quoted readings of the deciding facts and holds exactly the documents whose need depends on an open fact, with its properties and their scope (Section 3).
(2) CasePath, the artefact, with design knowledge from two design cycles, including a rule lint that needs no reference answers (Sections 4, 5 and 7).
(3) A pre-registered evaluation on sealed data against single-prompt and prior-work strategies, with checks added after review: an error decomposition, a budget-matched comparison, a clean-room re-derivation of the references and their sensitivity to the reading of the rules (Section 6).

We release the code, the research workflows, the synthetic data and the analysis [36], and the protocol of a planned study on the insurer's real files (Appendix B).

2 BACKGROUND AND RELATED WORK

CasePath combines rules executed by code, models that read text, and an explicit value for what is not known. Each paragraph names a line of work and what this article does differently; Table 1 compares the closest systems.

Design science. Design-science research builds an artefact for a class of problems, evaluates it [22, 47], and states design principles as its knowledge contribution [20]. Action design research builds and evaluates inside the organisation [53]; our evaluation so far uses synthetic claims outside the insurer.

Rules over text. Law and administration have executed rules since the British Nationality Act was written as a logic program [54]; Catala [40], the Decision Model and Notation standard (DMN) [43] and rules as code in government [41] continue the line. They take typed facts and treat a fact not supplied as false [49] or send the decision to a default branch; CasePath reads facts from text and keeps them unresolved unless a stated presumption applies. Neuro-symbolic systems have a model translate each problem into logic that a solver executes [16, 44, 45, 63], and legal systems compute an answer, such as a tax liability, from a complete representation of a case [23, 27, 31]; CasePath executes one reviewed workflow and must act before the case is complete. The closest system, LDPC, available as a preprint [11], answers yes/no questions as true, false or maybe and evaluates a model-written formula in strong Kleene logic, asking about a maybe; Proposition 3.2 shows what that evaluation costs when a workflow's branches meet again.

Case management and process models. Case management [42], declarative process models [48] and artifact-centric process models [25] let a case's data decide which tasks and documents apply. They assume the data are entered; CasePath reads them from the file and acts while some are unknown. Claims handling is such a knowledge-intensive process [6, 9, 10]; process research asks how language models can act within explicit constraints and keep people in control [8, 30, 34, 37, 57]; we study one recurring step, deciding which documents to request.

Agents that apply the rules. Agents usually follow a policy placed in their prompt, as τ-bench, FlowBench and procedure-following benchmarks evaluate [39, 60, 61]; Chain of Logic has the model evaluate a rule's elements [55], ReAct interleaves reasoning with tool calls [62], and self-consistency takes the majority of several runs [58]. In these designs the model applies the rules,

and repeated runs differ [1, 2]. Section 6 tests CasePath against a single prompt with the rules, self-consistency, a prompt that states the rules' semantics, and ReAct; it does not run Chain of Logic or the benchmarks' policy-in-prompt agents.

Reasoning over partial information. Three-valued model checking gives an answer that holds for every completion of a partial state space [3], partial models in software engineering reason over all concretisations of an uncertain model [13], and supervaluation [56] and certain answers over incomplete databases [26] do the same for sentences and queries. Exact deferral applies this idea to the documents a workflow needs; what is new is the application and the reconvergence case of Proposition 3.2, not the semantics.

Presumptions. Default logic draws conclusions that hold unless contradicted [50], and the Carneades model allocates the burden of proof in argument [19]. CasePath's presumptions are defaults of this kind, fixed per fact in the workflow with a source clause; the product does not reason about the burden of proof.

Asking, agreement and quotation. Selective prediction abstains when a model's confidence is low [4, 18, 29, 59], and interactive agents ask for missing information before they act [38, 64]. CasePath asks only when an open fact can change a document, a deterministic special case of the value of information [24]. Semantic entropy measures uncertainty from the agreement of sampled answers [14, 35], and citation benchmarks score whether cited sources support a model's statements [17]; CasePath makes agreement and a found quotation hard conditions, checked by code. The nearest agent design, a preprint that is not peer reviewed, keeps a ledger of evidence obligations in code, tied to their sources [32].

Legal benchmarks and supervision. LegalBench measures legal reasoning on 162 tasks [21]; our test measures how a design applies fixed rules to synthetic files, not legal reasoning. The EU AI Act classes AI systems for risk assessment and pricing in life and health insurance as high-risk [12]; evidence requests in legal-protection claims are not in that class, and we claim no regulatory standing for CasePath's records.

Reliance on advice. Experts follow wrong advice [15, 28], automation bias affects novices and experts alike [46], people turn away from algorithms after seeing them err [7], and hybrid intelligence asks how people and models complement each other [5, 52]. This article measures no reliance; the protocol in Appendix B is designed to.

3 THE DECISION RULE

The decision rule divides the work: a model reads the file and gives each fact a value, and code applies the workflow to those values. This section defines the rule and states four results with their scope; the proofs are in Appendix A.

We carry the mould claim of Section 1 throughout. One file says: "Our daughter has been coughing for weeks; the paediatrician suspects a link with the mould. I do not have a medical report yet." Another says: "…we do not know whether it has anything to do with the mould and have not seen a doctor." The first file needs the medical confirmation; the second leaves the need open, so the confirmation should be held and one question asked.

3.1 Workflows and facts

A workflow states an organisation's rules for one kind of claim. It names a finite set C of facts, the conditions its rules depend on, such as *health*: the mould has affected someone's health. A fact is true (T), false (F) or unresolved (U).

Table 1. Prior systems and CasePath. Yes, partly or no as stated in the cited work; –: not addressed there. Columns: (1) rules executed by code; (2) explicit unresolved value; (3) quotation required; (4) deferral exact. [a]The model writes the logic for each item. [b]For the facts that decide the checklist or the routing, within the scope stated in Section 3.

System	1	2	3	4
Rules as code, DMN, Catala [40, 41, 43, 54]	yes	no	no	no
Policy in the prompt [39, 60, 61]	no	–	–	–
Chain of Logic [55]	no	–	–	–
Logic-LM, LINC, SatLM, PAL [16, 44, 45, 63]	yes[a]	no	–	no
Statutes as programs, extracted facts [23, 27, 31]	yes	no	no	no
LDPC (preprint) [11]	yes[a]	yes	no	no
Case management, declarative and artifact-centric processes [25, 42, 48]	yes	–	–	–
Asking and abstaining [29, 38, 64]	no	partly	–	no
Evidence-obligation ledger (preprint) [32]	partly	yes	partly	–
CasePath (this article)	yes	yes	yes[b]	yes[b]

The workflow arranges steps in an acyclic graph $G = (N, E)$ with one entry step. Each step n has a condition c_n, and each transition $(u, n) \in E$ has a condition c_{un}. Each document type d is needed at a set of steps $L(d) \subseteq N$ under its own condition c_d. Conditions are formulas over facts built with *and*, *or* and *not*. We call such a workflow a *flow workflow*: its transitions only enable or branch to a step, with no relation that blocks, suppresses or waits for another step.

The output, a checklist, marks each document type request, hold, not needed or present, and lists as questions the facts on which held documents wait. Whether a document is on file is decided by two model judgments (Section 4.1); the propositions take it as given.

The product also has facts that take one of a list of values, a date or a text; for exact deferral such an open fact takes each listed value, or each value the conditions name and one for any other. In the tested workflows only the fact that selects the workflow is not two-valued. Completions treat facts as independent, so for related facts exact deferral can hold a document that every completion consistent with the relation needs.

3.2 Readings, admission and presumptions

A reading is one model pass over the file that answers every fact true or false with a quotation, open (the file says it is not known, quoting the words) or not stated. CasePath makes three readings by default, each a separate request. A quotation is *found* when, after soft hyphens are removed, words split by a hyphen at a line break are joined and white space is collapsed, it occurs letter for letter, with letter case kept, on one page of a file; failing that, a second match also equates the typographic variants of quotation marks, apostrophes, dashes, ligatures and spaces. Nothing else is folded, and a quotation shorter than eight characters is never found. A value whose quotation is not found, even after one request to quote again, counts as no value.

Table 2 gives the admission rule of the tested commit for the facts that decide the checklist or the routing: those in a transition, in a document's condition or in the workflow's selection condition. Code, not the model, applies it. Other facts are recorded from the first reading alone and do not enter the evaluation.

For the first file every reading gives *health* = T and quotes "Our daughter has been coughing for weeks; the paediatrician suspects a link with the mould", which is found, so T is admitted. For the

Table 2. Admission of a fact that decides the checklist or the routing, at commit 6a72159c. “Value” means a true or false answer, or a value, with its quotation found. A fact left unresolved holds the documents whose need it can change.

What the readings give	No presumption	Presumption of value v
Every reading gives the same value x	x is admitted	x is admitted
Readings give different values	unresolved; marked for a person	unresolved; marked for a person
Some readings give x, the others no value	unresolved	v if $x = v$; otherwise unresolved
No reading gives a value; one or more answer open with words found	unresolved	unresolved
No reading gives a value; none answers open with words found	unresolved	v

second every reading answers open, quoting “we do not know whether it has anything to do with the mould”.

Files are often silent on a fact, and silence is not doubt. A *presumption* states, with a clause from the organisation’s sources, what an unstated fact is taken to be; in our tests the clause is a silence clause we wrote, not the insurer’s. The checklist records a presumed value as a presumption. The tenancy workflow presumes that the mould has affected no one’s health when the file never mentions health. In the second file the readings answer open, so *health* stays U.

Admitted values and presumptions form an assignment $\nu : C \to \{\mathsf{T}, \mathsf{F}, \mathsf{U}\}$. A *completion* of ν is an assignment $\nu' : C \to \{\mathsf{T}, \mathsf{F}\}$ that agrees with ν wherever $\nu(f) \neq \mathsf{U}$.

3.3 Evaluation

Code evaluates conditions in strong Kleene logic [33]. With the order $\mathsf{F} < \mathsf{U} < \mathsf{T}$, *and* takes the minimum, *or* the maximum, and *not* swaps T and F. A step is active when it is the entry step or is reached from an active step through a transition that holds, and its own condition holds:

$$a(n) \;=\; \Big[\,\mathrm{entry}(n) \;\vee \bigvee_{(u,n)\in E} \big(a(u) \wedge \nu(c_{un})\big)\Big] \;\wedge\; \nu(c_n). \tag{1}$$

A document is needed when a step that needs it is active and its own condition holds:

$$r(d) \;=\; \Big[\bigvee_{n\in L(d)} a(n)\Big] \;\wedge\; \nu(c_d). \tag{2}$$

A document with $r(d) = \mathsf{F}$ is not needed; one with $r(d) = \mathsf{U}$ is held; one with $r(d) = \mathsf{T}$ is present if the file contains it and requested otherwise. We write r_ν when the assignment matters. Equations (1) and (2) are the semantics of flow workflows. All four tested workflows are flow workflows: under every complete assignment, the product’s own walk at the tested commit lists the documents these equations give (Appendix A says how we checked).

Proposition 3.1 (Soundness relative to admitted facts). *Let the workflow be a flow workflow over two-valued facts. If $r_\nu(d) \in \{\mathsf{T}, \mathsf{F}\}$, then $r_{\nu'}(d) = r_\nu(d)$ for every completion ν' of ν.*

Whatever the open facts turn out to be, a request stays needed and a document marked not needed stays unneeded. Learning an open fact’s value can turn a held document into a request or into not needed, but never reverses a request or a not needed. Two-valued evaluation lacks this property. Reading U as F, the closed-world assumption [49], drops documents that an open fact may require; reading U as T requests them too early.

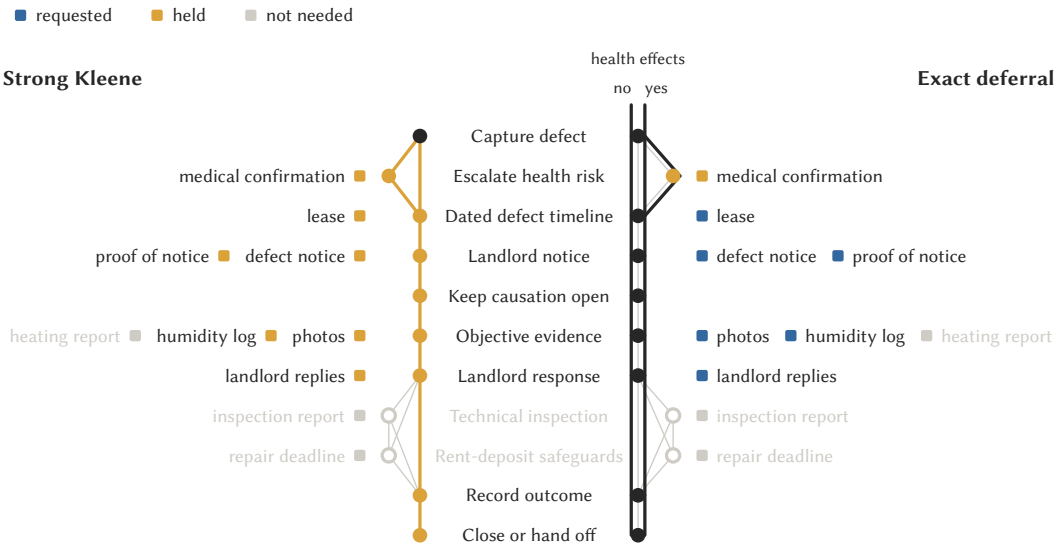


Fig. 2. The tenancy defect workflow with *health* unresolved, mould true, the other facts false and no document on file. Strong Kleene evaluation (left) holds every document after the branch whose own condition is not false; the heating report, the inspection and the repair deadline are not needed because their conditions are false. Exact deferral (right) holds only the medical confirmation, on which the two completions disagree. Because nothing is on file, the lease is requested here; in Figure 1 the claims attach the lease, so the product lists it as present and does not request it.

Proposition 3.2 (Strong Kleene evaluation over-holds). *Even in a flow workflow over two-valued facts, strong Kleene evaluation can hold a document that every completion of v needs.*

Strong Kleene evaluation forgets that two branches of a fork meet again. The tenancy defect workflow branches on *health*: to "Escalate health risk" if it is true, directly to "Dated defect timeline" otherwise, and the first leads to the second. With *health* = U, Equation (1) makes the timeline step U, and every later document whose own condition is not false is held. Under both completions the step is active (Figure 2).

3.4 Exact deferral

Exact deferral evaluates each document under every completion [56], as certain answers do for an incomplete database [26]:

$$\hat{r}(d) \;=\; \begin{cases} \mathsf{T} & \text{if } r_{v'}(d) = \mathsf{T} \text{ for every completion } v' \text{ of } v, \\ \mathsf{F} & \text{if } r_{v'}(d) = \mathsf{F} \text{ for every completion } v' \text{ of } v, \\ \mathsf{U} & \text{otherwise.} \end{cases} \tag{3}$$

The checklist is formed from $\hat{r}$ as from r. An open fact becomes a question when changing its value changes a held document or a step.

Proposition 3.3 (Exact deferral). *(a) For any workflow over two-valued facts whose need under a complete assignment is defined, exact deferral holds a document if and only if two completions of v disagree on it, and a document it requests or marks not needed has that status under every completion.*

(b) If the workflow is moreover a flow workflow, exact deferral never holds a document that strong Kleene evaluation requests or marks not needed.

Exact deferral holds what the open facts can change and nothing else. In Figure 2 it requests the lease, the defect notice and the later documents, holds the medical confirmation and asks whether the mould has affected anyone's health. It costs one evaluation per completion. Above eight open facts or 256 combinations of answers, the product returns the plain walk instead, which stops at the first transition whose condition is open and holds each document whose own condition is open. In Study 3, tenancy claims had at most 3 open facts and got exact deferral. Exact deferral never ran on theft: every theft text had 9–28 open facts and got the plain walk, which on the theft workflow gives what exact deferral gives (Appendix A). Study 1's research implementation fell back to strong Kleene evaluation above twelve open facts, which its claims never reached.

3.5 Reading errors

A reading can err in two ways: it can leave a settled fact unresolved or give it the wrong value. Let v^* be the true, complete assignment. An *unsafe error* is a request for a document that v^* does not need, or a document marked not needed that v^* needs.

Proposition 3.4 (Reading errors). *Let the workflow be over two-valued facts, and take whether a document is on file as given. (a) If $v(f) = v^*(f)$ for every fact with $v(f) \neq \mathsf{U}$, exact deferral makes no unsafe error. (b) If $v(f) \neq v^*(f)$ for some fact with $v(f) \neq \mathsf{U}$, every document whose $\hat{r}_v$ is T or F but differs from r_{v^*} is an unsafe error.*

An unresolved value costs holds and a question; a wrong value that passes admission produces an error the evaluation cannot detect. A presumption that the file's facts contradict is such a wrong value. Without a model, we computed exactly the expected outcome over the 150 tenancy claims of Study 1 when each fact is misjudged independently with probability 0.1, once with every misjudgement unresolved and once with every misjudgement the opposite value (Appendix A). Of the 1002 documents the claims need, unresolved values became 84.5 expected holds under exact deferral and no unsafe error; wrong values became 84.5 expected missed documents and 37.0 unnecessary requests.

Admission and agreement exist to turn wrong values into unresolved ones. In Study 3, 9 of 1344 tenancy fact values used by CasePath were admitted with a wrong value, and in each case all readings agreed (added after review; Supplement, Table S34).

4 CASEPATH

CasePath prepares a legal-protection claim from its files. It routes the claim to a workflow, reads the facts the workflow asks about, and returns a checklist, a brief and a reply draft for a case handler. Models read and write; code decides which documents are needed, requested or held; nothing is sent. The artefact of this article is the tested commit, `6a72159c` (source-tree hash `03048020. . .`).

4.1 The core and the rest

The claims of this article concern the *core*: several readings with exact quotation, admission by agreement, and code that executes the workflow with exact deferral (Figure 3). A *router*, a model agent, picks the workflow from routing facts it must quote; code selects the workflow those facts satisfy. Three *readings*, each a separate model request, answer every condition of the workflow, and Table 2 decides what is admitted. If the admitted values contradict the chosen workflow, code routes once more; a second contradiction hands the claim to a person. A *document finder* matches each document need to a file and quotes it, and a *second judgment*, one independent model call,

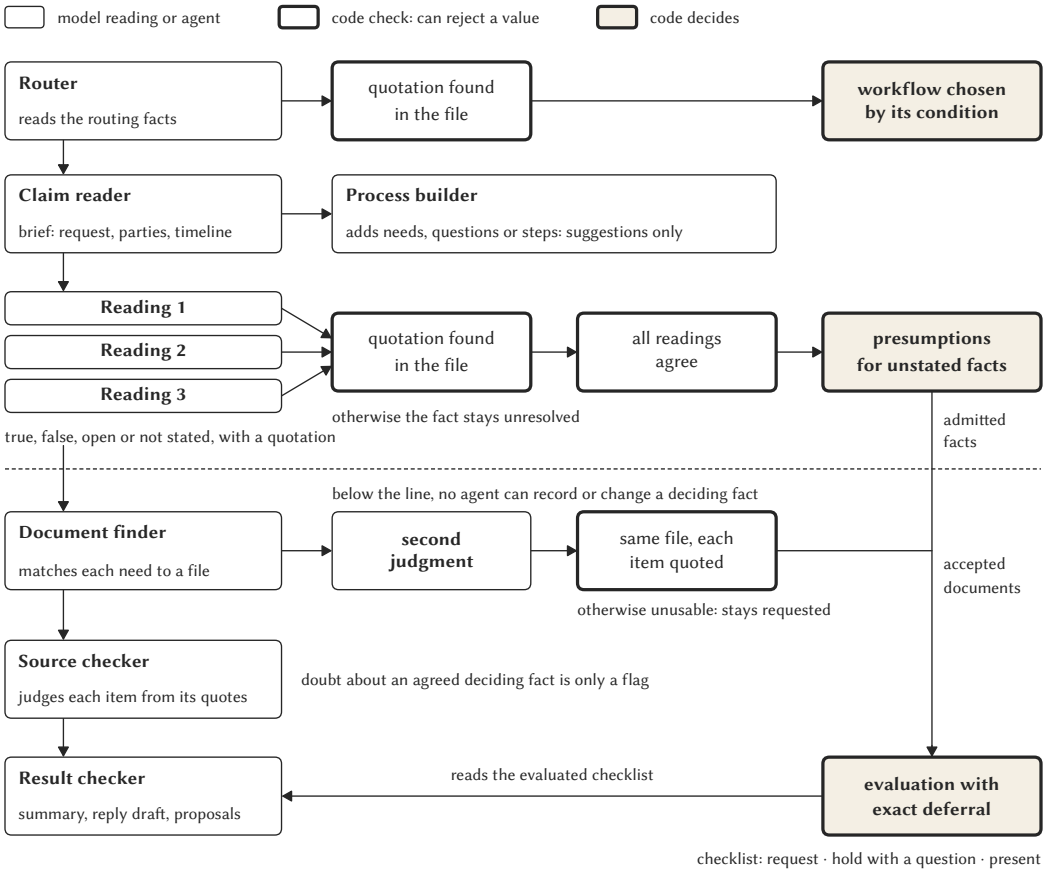


Fig. 3. The CasePath pipeline in the order a claim passes through it. Model readings and agents (thin boxes) propose values; code checks (thick boxes) reject a value whose quotation is not found or on which the readings differ; code alone decides the workflow, the presumptions and the checklist (tinted boxes), and below the dashed line no agent can change a deciding fact. Whether a document is present is decided by two model calls, the document finder and the second judgment, not by code. Drawn from the product's step names at commit 6a72159c.

must confirm the file and quote each item the need lists; otherwise the document stays requested. Evaluation is code: exact deferral over the admitted facts, the presumptions and the accepted documents.

The rest of the product is outside the claims. A *claim reader* writes a brief, a *process builder* may suggest up to six further needs, a *source checker* reviews quoted passages, and a *result checker* writes the open points and a reply draft. None of them can change a deciding fact once the readings are in. In Study 3, no need added by an agent entered a scored checklist, and dropping these agents produced no detectable difference in errors (Section 6.3). Supplement, Section S8 describes every step, what it may change, the Knowledge store and the parity command that proves two deployments run identical code.

Listing 1. The tenancy defect workflow (`defect_mold_heating.json`), trimmed with ellipses and with keys reordered for reading; nothing is added.

```
{"id": "defect_mold_heating", "applies_when": {"fact": "dispute_family", "is": "defect_mold_heating"}, ...
 "facts": [{"id": "health_effects", "type": "boolean", "question": "Does the claim allege effects of the defect
        on anyone's health?", ...}, ...],
 "documents": [{"id": "medical_confirmation", "required_when": {"fact": "health_effects", "is": true},
        "must_show": ["health findings", "causal limits"], ...}, ...],
 "steps": [{"id": "dh_intake", "next": [{"to": "dh_safety", "when": {"fact": "health_effects", "is": true},
        ...}, {"to": "dh_scope", "when": {"fact": "health_effects", "is": false}, ...}], ...}, ...],
 "presumptions": [{"fact": "health_effects", "applies": "unstated", "value": false, "source_ref":
        "policy-defect_mold_heating-v1#...clause-silence", ...}, ...], ...}
```

4.2 Workflows

A workflow is a data file in the format `casepath.workflow/1`. Every fact, step, document need and presumption must cite a source, or the file is rejected. Steps form a graph with conditions on transitions. A document need has a condition (`required_when`) and may list what the document must show (`must_show`). A presumption states what an unstated fact is taken to be and cites the clause that says so. Listing 1 shows the tenancy defect workflow of the running example.

4.3 The rule lint

Code executes faulty rules faithfully, so the rules need their own check. The rule lint flags a branch condition that the readings admit as true in most ordinary cases of its type: in at least half of the runs on ordinary texts, or with at least one reading true in three quarters of them. A condition meant to tell cases apart should not hold in nearly every case; when it does, it either belongs to the case type or is broader than its source. The lint reads only the case texts, the stored readings and the rule base, so it needs no reference answers; a person must decide each flag. It is a script released with the artefact (`analysis/rule_lint.py`), written after the test and not part of the tested commit. Section 6.7 reports what it found and its limits.

4.4 The tested configuration

Four settings define a run, and the parity output records each: the number of readings, three by default; the open rule, whose default `any` gives the rule of Table 2; the run scope, `full` by default; and a reasoning flag. With the flag on, as in Study 3, each call sends a reasoning effort and no temperature. The checklist-only setting (run scope `checklist`) skips the brief, the process builder, the result checker and the source checker's check of facts. Skipping that check changes no admitted value: for the facts that decide the checklist, the check sees only values all readings agree on, and it can only flag them. The setting also skips the document finder for a claim of one file, whose needs then count as missing; every theft text is such a claim.

5 HOW THE DESIGN WAS FOUND

The design came from two design cycles. The first built the core on synthetic data and evaluated it in three pre-registered studies. The second, now under way, carries it into the insurer's handling of real files.

Cycle 1: the core, on synthetic data. Two earlier configurations were each tested in a pre-registered study, and neither made fewer errors than Rules in context, a single prompt that applies the rules. The Study 1 configuration made one reading, had a second model call accept or reject each value, and evaluated a theft workflow that a model had extracted from insurance law and policy terms. With the frontier model it was no better than Rules in context: 38 against 22 wrong theft changes ($p = 0.211$) and 0.081 against 0.063 unsafe errors per tenancy claim ($p = 0.250$). Study 2 tested a

Table 3. Likely causes found in Study 1, the change each led to, and the evidence for the change. Counts are from Study 1 with the frontier model unless stated; Study 2 values are with the cost-efficient model. "Recomputed" marks evidence computed again from recorded readings, without new model calls. No row's evidence comes from a confirmatory test.

Cause in Study 1	Change	Evidence
Faulty rules: 37 of 38 wrong theft changes came from 5 of the 98 extracted rules.	A review of every rule against the passages it cites, after a repair without labels failed.	Study 2, new runs: 1.26 errors per theft pair with the label-free repair; recomputed: 1.17 with the original rules ($p = 0.562$). The review itself was evaluated on development data only.
Silence read as doubt: 638 of 735 unresolved values concerned facts that were false in the scenario, which the claims leave unmentioned.	Presumptions with a source clause.	Study 2, recomputed: 0.01 held needed documents per claim with presumptions, 0.84 without.
The checking call rejected 35 of 435 true values, all correct, and none of the 7 wrong ones.	A second reading that must agree; no checking call.	Study 2, recomputed by pairing each run's reading with another run's reading of the same text: errors per theft pair fell from 1.259 to 0.722 ($p = 0.004$, the smallest attainable).
Strong Kleene evaluation, the production engine's, held 48.6% of needed tenancy documents.	Exact deferral, which Study 1 had evaluated as a variant, became the evaluation.	Study 1, on the same readings: exact deferral held 43% (frontier) and 58% (cost-efficient) fewer, with no unnecessary request added.

second configuration on sealed new data and confirmed none of its four primary contrasts. Because code applied the rules, each error could be traced, post hoc, to a rule or a reading. Table 3 lists the likely causes and the change each led to. A first redesign, which withheld every rule an automatic check could not repair, was dropped on development data before Study 2. A repair of the rules without labels failed in Study 2 (1.26 errors per theft pair, against 1.17 with the original rules), and learning from handler corrections did not separate the configuration from Rules in context. Supplement, Sections S1 and S2 give both studies in full.

Four changes remain and form the decision rule of Section 3: a reviewed workflow, readings that must agree, presumptions and exact deferral. Two AI editor sessions checked each of the 98 extracted theft rules against its cited passages, independently; a change was adopted only when both made it, or one made it and the other approved it. They changed 9 rules and queued 14 for a jurist. No jurist has reviewed the rules, and the editors knew the theft conditions that Study 3 reuses. The joint effect of the four changes was measured only on development data before Study 3. An unplanned pilot on the public ShARC benchmark [51] compared the core with prompting on short single-rule items; no claim here rests on it (Supplement, Section S6).

Cycle 2: the deployment. In the second cycle the product's builders wrote a handler workflow for the insurer's mould cases from seven development files and the law. A protocol (Appendix B) will evaluate it. Its primary endpoint compares CasePath's output with what the insurer did next at every decision point of four sealed files; its secondary endpoint records what five of the insurer's claims experts do with the same output. The protocol is at version 1.0-rc and will be frozen on

20 November 2026. This article reports no result of the second cycle. Only counts will leave the insurer; files and texts stay with it.

6 EVALUATION (STUDY 3)

We tested CasePath at a frozen commit against designs that give a model the same files and the same rules, on sealed synthetic data under a plan fixed in advance. We call this test Study 3. It has two data sets, theft and tenancy, each with its own workflow and measures. Section 6.2 gives the registered outcomes. Sections 6.3 to 6.7 give checks added after an outside review of the results; they are not registered, and none changes a hypothesis's status.

6.1 Data, designs and freeze

The test data, `fresh2/`, were built and sealed before any design ran on them (Supplement, Section S9). Theft has 8 new stories: 72 *changed-fact pairs*, two texts that differ in one sentence that changes one fact, and 48 *unresolved-fact pairs*, in which the second text leaves that fact open. Tenancy has 96 new claims in 24 families, half in English and half in Swiss Standard German. Each tenancy dispute has a *defining event*, such as receiving a written rent-increase notification. The plan defines three subsets from the reference: 24 *denied-event* claims, which say the event has not happened; 9 *open-event* claims, which leave it open; and 63 *ordinary* claims, of which 23 leave another fact open. The test set was designed to include the denied- and open-event claims.

Three limits apply to every result. The tenancy references were produced by the engine CasePath implements, under a silence clause that we wrote and that every design receives. One model wrote every text, and a model of the same family checked them blind. The texts have 77–320 words, so the product's paths for long files were never exercised.

An AI agent session wrote the texts from a brief that held only the data design. It could read the repository, including the product's prompts and development results, so knowledge of the product may have shaped the texts in ways no check here can detect. The session that directed development, drafted the plan and chose the open rule had read the test set's design and counts, not its texts. Development effort was one-sided: CasePath was revised over three studies on development data, while no comparator text changed after 19:10 on 10 November. The open rule (Section 4.4) was set after a probe on 16 development claims, before the freeze: an independent AI review session of the product's code found a defect, and the directing author chose the rule `any`, overruling a criterion on wrong holds the author had set earlier. Table 4 gives the dates. At the time of writing, the product's owners have not confirmed the merge of the tested commit.

All designs reached the models through OpenRouter's chat-completions API: gpt-6-luna, the *cost-efficient model*, and gpt-6.1-terra, the *frontier model*; no snapshot is recorded. Table 5 lists the designs. A *case-run* is one run of one design on one case: a *claim-run* on tenancy, a *text-run* on theft; a *pair-run* covers both texts of a theft pair. Each design ran 3 times per case, except the frontier-model product, which ran once.

The measures are Study 2's. On tenancy, *strict unsafe errors per claim* count unnecessary requests, missed and handed-over required documents, premature requests and silent skips; *all deviations per claim* add held documents that should have been requested and *unneeded holds*, documents the reference neither requires nor holds. A design cannot look good by holding. On theft, *errors per pair* count the document changes between the two texts of a changed-fact pair that are wrong or missing; unresolved-fact pairs count pairs left otherwise unchanged.

6.2 Registered outcomes

Three of five hypotheses were confirmed, one failed and one was not tested (Table 6). The plan tested them in a fixed order, each at the 0.05 level only if all before it were confirmed: H1 theft and

Table 4. Timeline of Study 3, in UTC, from the commit dates of the repository and the run log.

Time	Event	Record
9 Nov, 22:37	`fresh/` sealed (data of Study 2, development data of Study 3)	commit 1c84cb3e
10 Nov, 17:05	`fresh2/` sealed	commit 5b45aa24
10 Nov, 19:17	first draft of the Study 3 plan	commit 71eaa9eb
10 Nov, 19:51	16 probe claims, runs and diagnosis committed	commit 6c7f4c89
10 Nov, 21:26	plan in final draft, five hypotheses	commit 984d2991
10 Nov, 21:51	hypotheses reordered (Section 6.2)	commit 482873e3
10 Nov, 21:53	open rule `any` committed to the product	commit a0d6211d
10 Nov, 22:18	product frozen at the tested commit	commit 6a72159c
10 Nov, 22:29	plan, specification, driver and analysis frozen	commit fd35e438
10 Nov, 22:30	run starts; all preflight checks pass	run log
11 Nov, 04:17	fifth invocation completes the run	run log
11 Nov, 04:18	analysis run once on the stored outputs; results committed	commit 48488138

Table 5. Designs of Study 3. Calls and cost per case-run, from the ledgers. All designs run on the cost-efficient model unless the name says frontier model. All receive the same files and catalogue and the reviewed rules, except the two original-rules designs.

Design	What it is	Calls (tenancy / theft)	USD per case (tenancy / theft)
CasePath	the product at the frozen commit	28.1 / 30.1	0.0134 / 0.0300
CasePath, checklist only	the same commit with run scope `checklist`	8.7 / 4.8	0.0030 / 0.0049
Core	two readings and exact deferral outside the product (theft only)	– / 2.0	– / 0.0028
Rules in context	one prompt applies the rules	1.0 / 1.0	0.0007 / 0.0007
Self-consistency	5 samples of Rules in context; majority per document	5.0 / 5.0	0.0034 / 0.0035
Model executes rules	one prompt that states the rules' semantics; the model names each condition's value and derives the documents	1.0 / 1.0	0.0007 / 0.0017
ReAct	a tool-using agent over the same workflow	8.1 / 2.9	0.0012 / 0.0019
Two-valued	CasePath's readings, unresolved taken as false or as true; no calls	– / –	– / –
Rules in context, original rules	the Study 1 rules (theft only)	– / 1.0	– / 0.0008
Rules in context, frontier model	the same prompt on the frontier model	1.0 / 1.0	0.0063 / 0.0041
original rules	the Study 1 rules on the frontier model (theft only)	– / 1.0	– / 0.0040
CasePath, checklist only, on the frontier model (one run)	the same commit and setting with the frontier model	7.4 / 4.2	0.0214 / 0.0386

H2 tenancy against Rules in context on the same model; H3 tenancy against Rules in context on the frontier model; H4 theft against each of Self-consistency, Model executes rules and ReAct, all three required; H5 theft non-inferiority to Rules in context on the frontier model. H1 to H4 use exact two-sided sign-flip tests over clusters, the 9 changed facts on theft and the 24 families on tenancy.

As the plan predicted, most of the tenancy difference lies in the 33 denied- and open-event claims: 0.47 strict unsafe errors per claim against 5.69 and 5.31 for the two single prompts. On the 63 ordinary claims, CasePath made 0.20 against 0.44 for the same model ($p = 0.0417$, which the plan did not predict) and 0.28 for the frontier model ($p = 0.409$, no difference). These subset rows are descriptive family means (Supplement, Table S16).

Table 6. Registered hypotheses in the fixed order, with three columns added after review and not registered. Values per pair (theft) or per claim (tenancy), as means of cluster values. Difference: CasePath minus comparator, with the registered 95% percentile cluster-bootstrap interval, which is too narrow with nine theft clusters (Supplement, Table S23). p: exact two-sided sign-flip test. Added columns: the outcome under the order of the plan's final draft (21:26), under the first draft's order (19:17), and under Holm's correction over the five hypotheses, from the registered p values.

						Registered	Added after review, not registered		
Hypothesis and comparator	CasePath	Comp.	Diff.	95% interval	p	Status	Order of 21:26	First draft	Holm (adj. p)
H1 Theft, Rules in context	0.58	1.74	−1.15	−1.65 to −0.64	0.0078	confirmed	confirmed	confirmed	confirmed (0.0156)
H2 Tenancy, Rules in context	0.31	2.14	−1.83	−2.72 to −0.99	0.0002	confirmed	confirmed	not in draft	confirmed (0.0006)
H3 Tenancy, Rules in context, frontier model	0.31	1.86	−1.55	−2.44 to −0.77	0.0008	confirmed	not tested	not tested[a]	confirmed (0.0024)
H4 Theft, Self-consistency	0.58	1.21	−0.63	−1.13 to −0.08	0.0625	not confirmed	not confirmed	not confirmed	not confirmed (0.0625)
Model executes rules	0.58	1.94	−1.36	−1.65 to −1.08	0.0039				
ReAct	0.58	1.69	−1.10	−1.57 to −0.57	0.0117				
H5 Theft, Rules in context, frontier model	0.58	0.95	not tested, because H4 failed			not tested	not tested	not tested	confirmed (0.0005)[b]

[a]The first draft had a non-inferiority form of H3, fourth in its order. [b]With the registered bootstrap non-inferiority p; with an exact sign-flip p the status is the same (Supplement, Section S13).

H4 required fewer theft errors than each of three prior-work strategies and failed against Self-consistency. The registered test gives each of the nine cluster differences a random sign; its smallest attainable p is 0.0039, and it returned 0.0625, although CasePath had the lower mean in all nine clusters. The differences are unequal in size, so sums as far from zero are not rare. An interval from inverting this test, added after review, runs from −1.25 to 0.08 and includes zero. With the eight stories as clusters instead, $p = 0.0156$; this changes no status. H5 was therefore not tested.

We changed the order of the hypotheses 38 minutes before the freeze, on development evidence. In the plan's final draft (21:26) the tenancy contrast against the frontier model came fifth. At 21:51 we moved it to third place, with the recorded reason that the contrasts development supported most strongly should come first. No design or scorer had then touched the sealed data. Under the order of 21:26, H3 would not have been tested. Under Holm's correction over the five hypotheses, which we did not register, H1, H2, H3 and H5 would pass and H4 would not. The registered outcome stands: three confirmed, one failed, one not tested.

6.3 All four cells

Across the four cells of model and data set, CasePath had fewer errors than the single prompt in three cells and no advantage in the fourth, theft on the frontier model (Table 7, Figure 4). On the cost-efficient model it had a lower mean error than every single-prompt and prior-work design in both data sets, and every such difference was significant except the theft difference from Self-consistency. On the frontier model the product ran once, as CasePath, checklist only, and those contrasts are descriptive. On tenancy it made 0.30 strict unsafe errors per claim against 1.86 for the frontier prompt ($p = 0.0012$). On theft it made 0.83 errors per pair against 0.95 for the frontier prompt with the reviewed rules ($p = 0.672$) and 0.67 with the original rules, the best single prompt on theft: no advantage.

Table 7. All four cells of model × data set. Top: errors per claim (tenancy, strict unsafe) or per pair (theft), means of cluster values; registered. Bottom, added after review: CasePath's errors split by the stage that caused them (Section 6.3); means over case-runs, which differ from the cluster means in the third decimal. The frontier columns decompose CasePath, checklist only, one run. Rule: errors with every fact at its designed value; reading: a wrong value admitted; hold: a stated fact left unresolved or an open fact given a value. –: not run.

	Tenancy		Theft	
Design	cost-efficient	frontier	cost-efficient	frontier
CasePath	0.31	–	0.58	–
CasePath, checklist only	0.34	0.30 (one run)	0.55	0.83 (one run)
Rules in context	2.14	1.86	1.74	0.95
Rules in context, original rules	–	–	1.47	0.67
Self-consistency	2.10	–	1.21	–
Model executes rules	0.83	–	1.94	–
ReAct	1.84	–	1.69	–
Core	–	–	0.77	–
CasePath's errors by stage (added after review, not registered)				
Rule	0.000	0.000	0.333	0.333
Routing	0.045	0.000	–	–
Document judgment	0.076	0.083	–	–
Reading	0.167	0.104	0.088	0.250
Hold	0.024	0.115	0.157	0.250
Other product steps	0.000	0.000	0.005	0.000
Measured (sum)	0.313	0.302	0.583	0.833

The decomposition, added after review, attributes each error to the stage that caused it. The run's stored state is replaced stage by stage, in a fixed order, by its designed value; the error that disappears at a stage is that stage's part, and what remains with every stage right is the rule's. The parts add up exactly, and their sum does not depend on the order. On tenancy the rule part is zero by construction, because the reference comes from the engine; most of the cost-efficient product's errors are readings that admitted a wrong value, mainly a denied defining event read as stated. On theft, the rules alone cause 0.333 errors per pair in both cells. The single prompts have no observable stage. Their tenancy errors are concentrated by kind: on denied-event claims, 4.431 to 5.917 unnecessary requests per claim for Rules in context on either model and Self-consistency; on open-event claims, 3.963 to 6.074 premature requests. Supplement, Section S13 gives the tables by subset.

The agents outside the core made no detectable difference in errors. CasePath, checklist only, made 0.34 against 0.31 strict unsafe errors per claim ($p = 0.5$) and 0.55 against 0.58 errors per pair ($p = 0.594$), at 22% and 16% of the full product's cost. The two settings shared cached responses, so this is no independent repetition.

6.4 What is held

CasePath's advantage lies in the files that leave a deciding fact open or deny the defining event (Table 8). The registered results show it in the holds. On theft, CasePath left 94% of unresolved-fact pairs otherwise unchanged, against 27–47% for every other design given the reviewed rules; the frontier prompt with the original rules reached 91%. On tenancy, CasePath held 97% of the documents the reference holds, against 48% and 35% for the single prompts. On tenancy, two-valued execution of CasePath's own readings made 1.27 (unresolved taken as false) and 1.51 (taken as

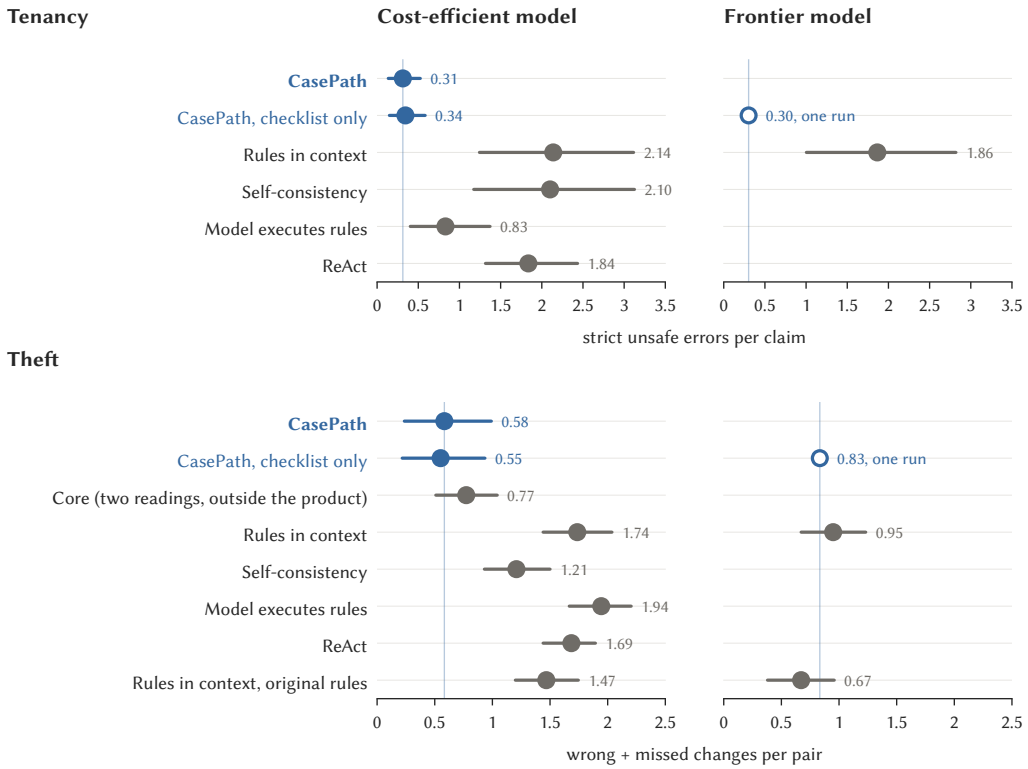


Fig. 4. All four cells of model × data set (registered results). Rows: tenancy, strict unsafe errors per claim; theft, wrong + missed changes per pair; one x scale per row. Columns: the cost-efficient and the frontier model. Filled dot: the design's mean over its clusters (24 families, 9 changed facts); bar: 95% percentile cluster-bootstrap interval of that mean, which under-covers with nine theft clusters. Open blue circle: CasePath, checklist only, on the frontier model, one run, without an interval. A row without a mark was not run on that model. Core: two readings and exact deferral run outside the product. Theft on the frontier model: 0.83 against 0.95 (reviewed rules) and 0.67 (original rules), no advantage.

true) strict unsafe errors per claim, against 0.31. On theft, exact deferral never ran, and "unresolved = false" gives the product's requests.

Splitting the tenancy claims by whether a fact is open or denied, added after review, locates the advantage. On the 56 claims with an open fact or a denied or open defining event, CasePath made 0.34 strict unsafe errors per claim; Rules in context made 2.32 on the cost-efficient model and 2.13 on the frontier model, and Self-consistency 2.24 (each $p \leq 0.0001$). Model executes rules, which states the rules' semantics in its prompt, made 0.90. On the 40 claims that state every fact, CasePath made 0.22 and the frontier prompt 0.19 ($p = 0.8984$): no advantage.

How much this matters depends on how many real files leave a deciding fact open, which we do not know. Figure 5 draws each design's expected errors against that share. CasePath's line lies below the frontier prompt's for any share above 0% to 1.7% (claim-run and family means). Counting unneeded holds as errors, the crossover lies at 4% to 7.2%. Against the other single-prompt designs the lines do not cross.

Table 8. What is held. Registered: theft, share of unresolved-fact pairs left otherwise unchanged; tenancy, share of reference-held documents held, and unneeded holds per claim. Added after review, not registered: tenancy strict unsafe errors per claim on the 40 claims that state every fact (S) and on the 56 claims with an open fact or a denied or open defining event (O), as family means. –: not run or not applicable.

	Registered			Added after review	
	Theft	Tenancy		Tenancy, strict	
Design	unchanged (%)	held (%)	unneeded	S	O
CasePath	94	97	0.33	0.22	0.34
CasePath, checklist only	95	100	0.30	0.24	0.40
Rules in context	27	48	0.17	0.40	2.32
Self-consistency	45	55	0.18	0.33	2.24
Model executes rules	27	86	0.58	0.34	0.90
ReAct	47	55	0.09	0.97	2.11
Two-valued, unresolved = false	94	0	0.00	0.33	1.81
Two-valued, unresolved = true	7	0	0.00	0.24	2.07
Core	85	–	–	–	–
Rules in context, original rules	31	–	–	–	–
Rules in context, frontier model	45	35	0.18	0.19	2.13
original rules	91	–	–	–	–
CasePath, checklist only, frontier (one run)	75	87	0.39	0.05	0.51

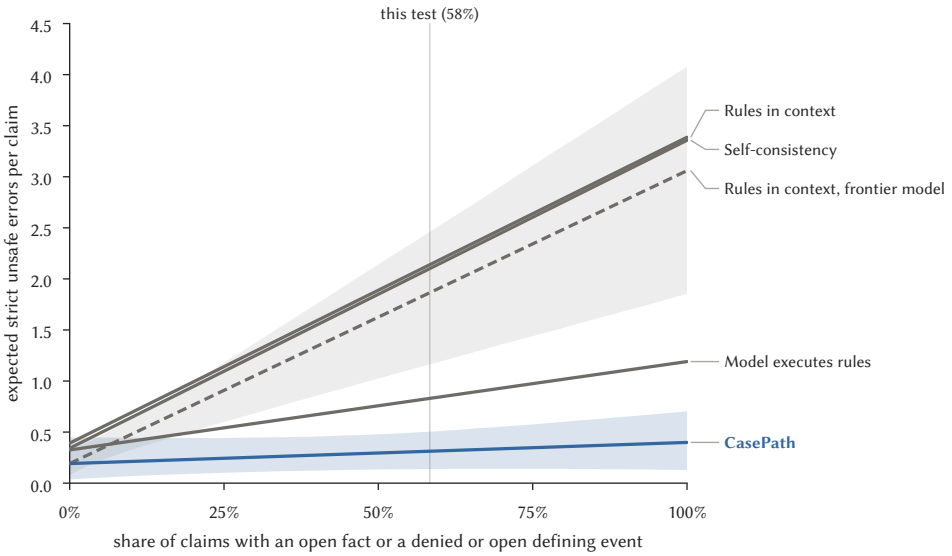


Fig. 5. Expected strict unsafe errors per claim against the share of claims with an open fact or a denied or open defining event (added after review, not registered). Each line runs from a design's mean on the claims that state every fact to its mean on the other claims (claim-run means). Bands: 95% family-bootstrap intervals for CasePath and the frontier single prompt. Vertical line: this test's share (58%). The share in real files is not known to us.

Holding has a cost. CasePath held 0.33 documents per claim that the reference neither requires nor holds, against 0.17 and 0.18 for the single prompts, most of them on denied-event claims

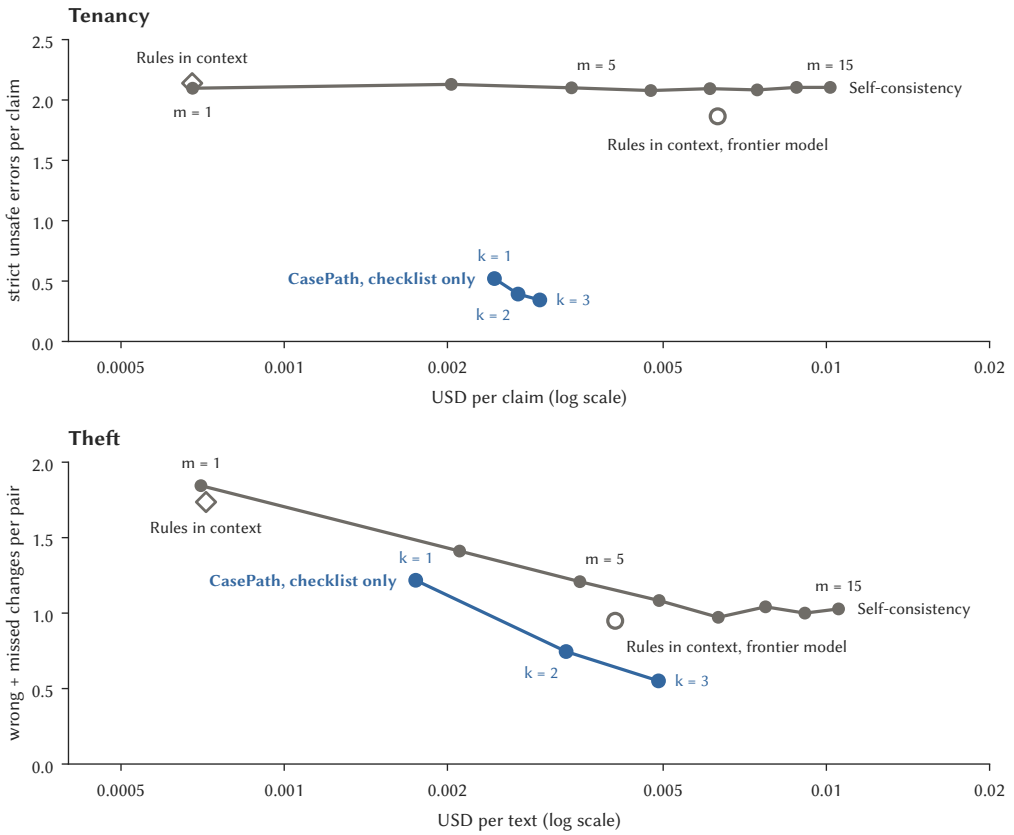


Fig. 6. Errors against cost per case, log cost axis (added after review, not registered). Grey line: Self-consistency with $m = 1$ to 15 samples of Rules in context ($m = 1$ is the mean of the 15 stored single samples). Blue: CasePath, checklist only, with $k = 1, 2, 3$ readings. Hollow diamond: Rules in context as registered, which is not the same quantity as $m = 1$. Open circle: Rules in context on the frontier model. Points are means, without intervals. More samples of the same prompt stay near 2.10 on tenancy and level off near 1.00 on theft.

where the readings did not all agree on the denial. Counting these as errors, CasePath made 0.73 deviations per claim, against 2.33 ($p = 0.001$) and 2.09 ($p = 0.0023$). Each unneeded hold is a question to the claimant that the file had already answered.

6.5 The same budget

More samples of the same prompt did not buy what the design buys (Figure 6; added after review, not registered). We recomputed CasePath, checklist only, with its first $k = 1$, 2 and 3 stored readings, and Self-consistency with $m = 1$ to 15 of its stored samples, without new model calls. On tenancy, Self-consistency stayed near 2.10 strict unsafe errors per claim from $m = 1$ to $m = 15$ (2.10). On theft it fell from 1.84 to about 1.00 errors per pair and levelled off from $m = 9$. CasePath fell from 0.52 to 0.34 (tenancy) and from 1.22 to 0.55 (theft) as readings were added.

At a matched budget, the lead was significant on tenancy and not on theft. The cheapest Self-consistency that cost at least as much as CasePath, checklist only, used $m = 5$ samples on

tenancy and $m = 7$ on theft. CasePath's difference from it was −1.76 strict unsafe errors per claim ($p = 0.0006$) and −0.53 errors per pair ($p = 0.1328$, not significant). Fifteen samples did not reach the full product's cost, so no contrast matches it.

More readings lower strict errors and raise unneeded holds. From $k = 1$ to $k = 3$ on tenancy, unneeded holds rose from 0.003 to 0.302 per claim, and all deviations from 0.55 to 0.74. Each added reading can disagree, and a disagreement holds.

The full product costs several times a single prompt. Per tenancy claim and theft text, it cost USD 0.013 and 0.030, 2.1 and 7.4 times the frontier single prompt; checklist only cost 0.003 and 0.005. In the test, with 20 cases in parallel, a full case-run took a median of 204 and 287 s, checklist only 33 and 87 s, and the frontier single prompt 14 and 10 s. We did not measure a handler's time.

6.6 Does the result depend on our reading of the rules?

Two blind evaluators reproduced the tenancy references exactly. Under a rejected reading of the rule text, the product's unchanged workflow can lose its lead; once the workflow encodes the reading, the lead holds under every reading we can read as evidence (Table 9; added after review, not registered). The tenancy references come from the engine CasePath implements, so a reviewer may object that the test rewards the product's own reading of the rules. We answered in three steps.

First, a clean room. Two AI evaluator sessions, which never saw the engine, its outputs or the reference, each wrote an evaluator from the rule text and the designed fact states alone. Their references agreed with the registered one on 1024 of 1024 and 1024 of 1024 document decisions. No person checked their work. After delivering its hashed reference, one session listed a scratch directory and saw the names of other files of this work; by its report it opened none.

Second, the product's unchanged workflow scored against references written under four readings that the evaluators had rejected. Under A1, "a denied defining event does not stop the process", the tenancy result reverses: the difference from Rules in context becomes +0.55 ($p = 0.1348$) and from the frontier prompt +0.21 ($p = 0.5134$), and two all-deviation contrasts reverse significantly. With all four readings together, H2's contrast reverses significantly (+0.94, $p = 0.0234$). Under A2, A3 and A4 the result holds. This step measures the workflow: a workflow written under the wrong reading is wrong in every file the reading affects.

Third, the workflow amended to each reading, with the checklist recomputed from the stored readings and no model calls. The unamended replay reproduces 285 of 286 stored checklists. CasePath then keeps a significant lead over both single prompts under every reading. It is smaller under A1 and A2. Under A1 the lead consists of the open-event claims; under A2, of the denied-event claims. The baselines were not re-run, and their prompts carried the registered rule text. Under A1 this favours them, because a prompt that ignored the registered stop is scored as right. Under A3 and "all four" it works against them, because they were never told the reading. Those two rows are upper bounds; only A1, A2 and A4 are read as evidence.

6.7 Rules as the limit, and a lint that finds them

On theft with the frontier model the limit is the rules, and the lint found the faulty conditions but its remedy did not survive out of sample (Table 10; added after review, not registered). The decomposition splits that cell's 0.833 errors per pair into 0.333 from the rules, 0.250 from readings and 0.250 from holds. With every fact read correctly, the reviewed rules still give 0.333. Most of the rest comes from two over-broad conditions read literally. One requires a third party at fault: the frontier model admitted it in every ordinary text, on a sentence excluding any third party "apart from the unidentified thief". The other concerns traces of the theft that were altered, whose value changed between the two texts of a pair.

Table 9. Sensitivity to the reading of the tenancy rule text (added after review, not registered). Agree: document decisions of the reference under the reading that equal the registered reference (of 1024). Strict unsafe errors per claim, family means: CasePath with the registered workflow (unchanged) and with the workflow amended to the reading (matched); the single prompts' stored checklists scored against the same reference. H2 and H3: CasePath minus Rules in context on the cost-efficient and the frontier model, with exact sign-flip p. A1: a denied defining event does not stop the process; A2: an open defining event is presumed to have occurred; A3: optional documents are requested; A4: a family-home condition excludes arrears. Rows A3 and "all four" are upper bounds for the matched workflow (see text).

		CasePath		Rules in context		Unchanged workflow		Matched workflow	
Reading	Agree	unchanged	matched	cost-eff.	frontier	H2 (p)	H3 (p)	H2 (p)	H3 (p)
Registered	1024	0.31	0.31	2.14	1.86	−1.83 (0.0002)	−1.55 (0.0008)	−1.83 (0.0002)	−1.55 (0.0008)
A1	865	1.37	0.16	0.82	1.16	0.55 (0.1348)	0.21 (0.5134)	−0.66 (0.0002)	−1.00 (0.0014)
A2	961	0.29	0.29	1.72	1.30	−1.43 (0.0002)	−1.01 (0.0020)	−1.43 (0.0002)	−1.01 (0.0020)
A3	952	1.06	0.34	2.88	2.61	−1.82 (0.0002)	−1.55 (0.0008)	−2.54 (< 0.0001)	−2.27 (< 0.0001)
A4	1019	0.34	0.31	2.17	1.90	−1.83 (0.0002)	−1.55 (0.0008)	−1.86 (0.0001)	−1.59 (0.0007)
All four	702	2.38	0.14	1.44	1.63	0.94 (0.0234)	0.75 (0.0472)	−1.30 (< 0.0001)	−1.49 (< 0.0001)

Table 10. Theft, "a flagged condition is treated as open", recomputed from the stored readings without model calls (added after review, not registered). In sample: flags from the same texts. Out of sample: flags from development runs only. Rule: the part of the errors the rules cause with every fact right. Held: documents held in every ordinary text. Open: open facts that decide a document, per ordinary text, each a question to the claimant. p: exact sign-flip test over nine clusters against the frontier prompt with the reviewed rules (0.95) and with the original rules (0.67).

Model	Flags	Errors	Rule	Unchanged	Held	Open	p reviewed	p original
Frontier (one run)	none (tested)	0.833	0.333	0.750	9	13.63	0.6719	0.4297
	in sample	0.528	0.444	1.000	14	19.00	0.1406	0.4531
	out of sample	0.778	0.444	0.917	12	18.63	0.5234	0.5742
Cost-efficient	none (tested)	0.579	0.333	0.944	6	11.88	–	–
	in sample	0.574	0.333	0.951	8	15.88	–	–

The lint flagged these conditions without reference answers. A mutation test checked it on tenancy, with one model and one run per claim, for USD 0.15. We widened 6 conditions to general circumstances, and the lint flagged 6. 3 of them were flagged already in their original wording, so the informative count is 3 of 3. 3 of 5 untouched conditions were flagged in both arms. On the Study 3 tenancy claims the lint flags 8 of 10 conditions with the cost-efficient model's readings and 9 of 10 with the frontier model's, without any mutation. Those claims were built with several true branch facts each and are not ordinary cases. The lint therefore detects blunt over-breadth; its false-alarm rate is unknown and needs files in which a condition is known not to apply.

Treating a flagged condition as open helped in sample and not out of sample. With the frontier model's own flags, its theft errors fell from 0.833 to 0.528 per pair, below both frontier prompts, and every unresolved-fact pair stayed otherwise unchanged. With the cost-efficient model they barely moved (0.579 to 0.574). The gain has a price: the rule part rises from 0.333 to 0.444, and more documents are held in every ordinary text, each with a question to the claimant. With flags computed from development runs only, the frontier errors fell only to 0.778, still above the frontier prompt with the original rules. The development flags miss the traces condition, which carried most of the in-sample gain. No paired difference is significant (p from 0.1406 to 0.6719 over nine clusters). The remedy does not survive out of sample, so we present the lint as a diagnostic, not as

a tested remedy. We made no new paid run for this cell: with nine clusters, a difference of 0.528 against 0.67 cannot be confirmed.

Deciding a flag needs a jurist. Two blind AI editor sessions reviewed the lint's flags under the four-eyes rule of the first review. They could not agree on narrowing the third-party condition: one rejected it, because its source passage states no exclusion. No jurist has reviewed the rules.

7 DISCUSSION

We state the design knowledge as three principles. Each comes with its evidence, marked registered or added after review, and with the boundary condition under which it holds.

Principle 1: execute in code the rules that stop or hold the checklist, and check the rules. Registered: on the 24 denied-event and 9 open-event tenancy claims, CasePath made 0.61 and 0.26 strict unsafe errors per claim, Rules in context 5.68 and 4.63, and Model executes rules 1.94 and 1.04. Stating the rules' semantics in the prompt closed most of the gap, and executing them in code closed most of the rest. *Boundaries:* the gain grows with the share of files that leave a deciding fact open or deny the defining event (Figure 5); on files that state every fact there was no advantage over the frontier prompt. Code also executes faulty rules faithfully. On theft the rules alone cause 0.333 errors per pair, and the lint finds over-broad conditions but cannot decide them (Section 6.7).

Principle 2: give an open fact its own value, and evaluate every completion. Registered: CasePath held 97% of the documents the tenancy references hold, against 48% for Rules in context. Two-valued execution of its own readings made 1.27 and 1.51 strict unsafe errors per claim, against 0.31. Added after review: against strong Kleene evaluation of the same facts, exact deferral made no fewer strict unsafe errors (0.25 against 0.23) but fewer deviations (0.65 against 1.31 per claim), because it held fewer documents. *Boundaries:* exact deferral never ran on theft, so the principle is supported on tenancy only, within the scope of Section 3. Its price is unneeded holds.

Principle 3: admit only agreeing readings with quotations found in the file. No registered test isolates this principle. Added after review: of 1344 tenancy fact values, 9 admitted values were wrong. With one reading instead of three, strict unsafe errors rose from 0.34 to 0.52 per claim and theft errors from 0.55 to 1.22 per pair. *Boundaries:* in every wrong admission all readings agreed, so readings that share one model and prompt miss a shared misreading. Each added reading also raises unneeded holds. Whether a document is present is a model judgment outside this rule.

For an organisation. Someone must write the workflows and their presumptions, cite a source for each, and have the rules reviewed. Here AI editor sessions reviewed the theft rules, and three of their questions await a jurist. The lint points such a review at conditions that hold in nearly every case, without reference answers. An auditor gets each deciding value with its quotation, each presumption with its clause, each hold with its question, and the hash of the code that ran. A hold costs a handler a question to the claimant and a wait for the answer; in this test CasePath asked such questions more often than needed.

7.1 Limitations

The tenancy references come from the engine CasePath implements, under one reading of a rule text and a silence clause that we wrote. Two blind evaluators reproduced them exactly, but under the reading that a denied event does not stop the process, the product's unchanged workflow loses its lead (Section 6.6). The matched-workflow replay did not re-run the baselines. The data are synthetic and short (77–320 words), written by a model session that could read the product's prompts, so long files and real intake were never tested. The theft rules were reviewed by AI editor sessions that knew the test conditions; no jurist has reviewed them, and theft reuses Study 1's

conditions. The clean-room evaluators and the lint's editors were AI sessions as well, and no person checked their work. Wrong presumptions were tested only by rescoring with some unmentioned facts made true, in which every design's errors rose by about as much (Supplement, Section S12.2).

The statistics have limits of their own. H3 was tested only because we reordered the hypotheses 38 minutes before the freeze, on development evidence. With nine theft clusters, H4's failure does not show the designs equal, and the registered bootstrap intervals are too narrow. On the frontier model the product ran once, without a hypothesis; on theft it made 0.83 errors per pair, against 0.67 for the frontier prompt with the original rules, the best single prompt on theft. Every check in Sections 6.3 to 6.7 was decided after the results were known; the decomposition maps designed theft facts to the rules' conditions by hand, and two of those mappings are uncertain.

The design has costs. The full product cost several times a single prompt and took minutes per case. It held more documents than needed, and more readings hold more. The agents outside the core, the brief, the process builder and the reply draft, made no detectable difference in errors; they may be worth their cost to a handler, which this article does not measure. The lint's false-alarm rate is unknown, and its remedy failed out of sample.

The run had irregularities: four of five invocations of the driver stopped early, two of them by failures of the environment, and 50 comparator outputs failed and were scored as empty checklists (Appendix C). No result comes from real files or from claims experts yet. The deployment study's protocol (Appendix B) compares the product with what the insurer did next on four sealed files; it is at version 1.0-rc, to be frozen on 20 November 2026, and its stated limits include files seen in development and a handler workflow written by the product's builders.

8 CONCLUSION

In CasePath, a model reads each deciding fact several times and quotes the file; code admits agreeing values, executes the organisation's rules and holds what the file leaves open. In a pre-registered test of a frozen commit on synthetic claims, the design held what single prompts with the same rules did not hold. On the cost-efficient model it had a lower mean error than every single-prompt and prior-work design in both data sets. The registered tests confirmed this against the single prompt in both and against two of three prior-work strategies on theft; against self-consistency the theft lead (0.58 against 1.21) gave $p = 0.0625$, and that hypothesis failed. On the frontier model, run once, it made fewer tenancy errors and had no advantage on theft (0.83 errors per pair against 0.95 and, with the original rules, 0.67), where the rules set the limit. The theft rules had been reviewed by AI sessions that knew the test conditions, and the tenancy references come from the engine CasePath implements; under one rejected reading of the rule text the product's unchanged workflow loses its tenancy lead. Whether it helps case handlers on real files is the question of the deployment study.

ACKNOWLEDGMENTS

We thank the three anonymous reviewers of the conference version for detailed criticism that shaped this revision.

REFERENCES

[1] Berk Atıl, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. 2025. Non-Determinism of "Deterministic" LLM System Settings in Hosted Environments. In *Proceedings of the 5th Workshop on Evaluation and Comparison of NLP Systems*, Mousumi Akter, Tahiya Chowdhury, Steffen Eger, Christoph Leiter, Juri Opitz, and Erion Çano (Eds.). Association for Computational Linguistics, Mumbai, India, 135–148. https://doi.org/10.18653/v1/2025.eval4nlp-1.12

[2] Andrew Blair-Stanek and Benjamin Van Durme. 2025. LLMs Provide Unstable Answers to Legal Questions. In *Proceedings of the Twentieth International Conference on Artificial Intelligence and Law (ICAIL 2025)*. Association for Computing Machinery, 425–429. https://doi.org/10.1145/3769126.3769245

[3] Glenn Bruns and Patrice Godefroid. 1999. Model Checking Partial State Spaces with 3-Valued Temporal Logics. In *Computer Aided Verification (11th International Conference, CAV'99, Proceedings) (Lecture Notes in Computer Science)*, Nicolas Halbwachs and Doron Peled (Eds.). Springer, Berlin, Heidelberg, 274–287. https://doi.org/10.1007/3-540-48683-6_25

[4] C. K. Chow. 1970. On Optimum Recognition Error and Reject Tradeoff. *IEEE Transactions on Information Theory* 16, 1 (1970), 41–46. https://doi.org/10.1109/TIT.1970.1054406

[5] Dominik Dellermann, Philipp Ebel, Matthias Söllner, and Jan Marco Leimeister. 2019. Hybrid Intelligence. *Business & Information Systems Engineering* 61, 5 (2019), 637–643. https://doi.org/10.1007/s12599-019-00595-2

[6] Claudio Di Ciccio, Andrea Marrella, and Alessandro Russo. 2015. Knowledge-Intensive Processes: Characteristics, Requirements and Analysis of Contemporary Approaches. *Journal on Data Semantics* 4, 1 (2015), 29–57. https://doi.org/10.1007/s13740-014-0038-4

[7] Berkeley J. Dietvorst, Joseph P. Simmons, and Cade Massey. 2015. Algorithm Aversion: People Erroneously Avoid Algorithms after Seeing Them Err. *Journal of Experimental Psychology: General* 144, 1 (2015), 114–126. https://doi.org/10.1037/xge0000033

[8] Marlon Dumas, Fabiana Fournier, Lior Limonad, Andrea Marrella, Marco Montali, Jana-Rebecca Rehse, Rafael Accorsi, Diego Calvanese, Giuseppe De Giacomo, Dirk Fahland, Avigdor Gal, Marcello La Rosa, Hagen Völzer, and Ingo Weber. 2023. AI-augmented Business Process Management Systems: A Research Manifesto. *ACM Transactions on Management Information Systems* 14, 1 (2023), 1–19. https://doi.org/10.1145/3576047

[9] Martin Eling and Martin Lehmann. 2018. The Impact of Digitalization on the Insurance Value Chain and the Insurability of Risks. *The Geneva Papers on Risk and Insurance – Issues and Practice* 43, 3 (2018), 359–396. https://doi.org/10.1057/s41288-017-0073-0

[10] Martin Eling, Davide Nuessle, and Julian Staubli. 2022. The Impact of Artificial Intelligence along the Insurance Value Chain and on the Insurability of Risks. *The Geneva Papers on Risk and Insurance – Issues and Practice* 47, 2 (2022), 205–241. https://doi.org/10.1057/s41288-020-00201-7

[11] Kyle Erwin, Guy Axelrod, Maria Chang, Achille Fokoue, Maxwell Crouse, Soham Dan, Tian Gao, Rosario Uceda-Sosa, Ndivhuwo Makondo, Naweed Khan, and Alexander Gray. 2025. Few-shot Policy (de)composition in Conversational Question Answering. arXiv:2501.11335. https://arxiv.org/abs/2501.11335

[12] European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying Down Harmonised Rules on Artificial Intelligence and Amending Regulations (EC) No 300/2008, (EU) No 167/2013, (EU) No 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and Directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (Artificial Intelligence Act). Official Journal of the European Union, L series, 2024/1689, 12 July 2024. https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng

[13] Michalis Famelis, Rick Salay, and Marsha Chechik. 2012. Partial Models: Towards Modeling and Reasoning with Uncertainty. In *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 573–583. https://doi.org/10.1109/ICSE.2012.6227159

[14] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting Hallucinations in Large Language Models Using Semantic Entropy. *Nature* 630, 8017 (2024), 625–630. https://doi.org/10.1038/s41586-024-07421-0

[15] Andreas Fügener, Jörn Grahl, Alok Gupta, and Wolfgang Ketter. 2021. Will Humans-in-the-Loop Become Borgs? Merits and Pitfalls of Working with AI. *MIS Quarterly* 45, 3 (2021), 1527–1556. https://doi.org/10.25300/MISQ/2021/16553

[16] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 10764–10799. https://proceedings.mlr.press/v202/gao23f.html

[17] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398

[18] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. https://papers.nips.cc/paper_files/paper/2017/file/4a8423d5e91fda00bb7e46540e2b0cf1-Paper.pdf

[19] Thomas F. Gordon, Henry Prakken, and Douglas Walton. 2007. The Carneades Model of Argument and Burden of Proof. *Artificial Intelligence* 171, 10–15 (2007), 875–896. https://doi.org/10.1016/j.artint.2007.04.010

[20] Shirley Gregor and Alan R. Hevner. 2013. Positioning and Presenting Design Science Research for Maximum Impact. *MIS Quarterly* 37, 2 (2013), 337–355. https://doi.org/10.25300/MISQ/2013/37.2.01

[21] Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Aditya K, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam

Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny Gandhi, Tom Zur, Varun Iyer, and Zehua Li. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Neural Information Processing Systems Foundation, Inc., 44123–44279. https://doi.org/10.52202/075280-1915

[22] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. 2004. Design Science in Information Systems Research. *MIS Quarterly* 28, 1 (2004), 75–106. https://doi.org/10.2307/25148625

[23] Nils Holzenberger and Benjamin Van Durme. 2023. Connecting Symbolic Statutory Reasoning with Legal Information Extraction. In *Proceedings of the Natural Legal Language Processing Workshop 2023*, Daniel Preoţiuc-Pietro, Catalina Goanta, Ilias Chalkidis, Leslie Barrett, Gerasimos Spanakis, and Nikolaos Aletras (Eds.). Association for Computational Linguistics, Singapore, 113–131. https://doi.org/10.18653/v1/2023.nllp-1.12

[24] Ronald Howard. 1966. Information Value Theory. *IEEE Transactions on Systems Science and Cybernetics* 2, 1 (1966), 22–26. https://doi.org/10.1109/TSSC.1966.300074

[25] Richard Hull. 2008. Artifact-Centric Business Process Models: Brief Survey of Research Results and Challenges. In *On the Move to Meaningful Internet Systems: OTM 2008 (OTM 2008 Confederated International Conferences, Proceedings, Part II) (Lecture Notes in Computer Science)*, Robert Meersman and Zahir Tari (Eds.). Springer, Berlin, Heidelberg, 1152–1163. https://doi.org/10.1007/978-3-540-88873-4_17

[26] Tomasz Imieliński and Witold Lipski, Jr. 1984. Incomplete Information in Relational Databases. *J. ACM* 31, 4 (1984), 761–791. https://doi.org/10.1145/1634.1886

[27] William Jurayj, Nils Holzenberger, and Benjamin Van Durme. 2026. Language Models and Logic Programs for Trustworthy Tax Reasoning. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 45 (2026), 38688–38698. https://doi.org/10.1609/aaai.v40i45.41212

[28] Ekaterina Jussupow, Kai Spohrer, Armin Heinzl, and Joshua Gawlitza. 2021. Augmenting Medical Diagnosis Decisions? An Investigation into Physicians' Decision-Making Process with Artificial Intelligence. *Information Systems Research* 32, 3 (2021), 713–735. https://doi.org/10.1287/isre.2020.0980

[29] Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective Question Answering under Domain Shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 5684–5696. https://doi.org/10.18653/v1/2020.acl-main.503

[30] Timotheus Kampik, Christian Warmuth, Adrian Rebmann, Ron Agam, Lukas N. P. Egger, Andreas Gerber, Johannes Hoffart, Jonas Kolk, Philipp Herzig, Gero Decker, Han van der Aa, Artem Polyvyanyy, Stefanie Rinderle-Ma, Ingo Weber, and Matthias Weidlich. 2025. Large Process Models: A Vision for Business Process Management in the Age of Generative AI. *KI – Künstliche Intelligenz* 39, 2 (2025), 81–95. https://doi.org/10.1007/s13218-024-00863-8

[31] Manuj Kant, Manav Kant, Marzieh Nabi, Preston Carlson, and Megan Ma. 2024. Equitable Access to Justice: Logical LLMs Show Promise. arXiv preprint arXiv:2410.09904. arXiv:2410.09904 [cs.AI] https://arxiv.org/abs/2410.09904

[32] Myung Ho Kim. 2026. Evidence-Obligation Pool-Gated Retrieval: Stable Multi-Cycle Retrieval via Evidence Ledger and Warrant Judge. Preprints.org, version 1, posted 15 July 2026. https://doi.org/10.20944/preprints202607.1060.v1 Preprint (not peer reviewed).

[33] Stephen Cole Kleene. 1952. *Introduction to Metamathematics.* D. Van Nostrand Company.

[34] Fabian König, Andreas Egger, Wolfgang Kratsch, Maximilian Röglinger, and Niklas Wördehoff. 2025. Unstructured Data in Process Mining: A Systematic Literature Review. *ACM Transactions on Management Information Systems* 16, 3 (2025), 1–34. https://doi.org/10.1145/3727148

[35] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=VD-AYtP0dve

[36] Navish Kumar, Tobias Hoesli, Aydin Javadov, and Joseph Ollier. 2026. CasePath: Source Code, Datasets, Experiment Harness and Analysis Plan. GitHub repository. https://github.com/KumarNavish/casepath Experiments and paper source in `research/tmis2026`.

[37] Dongchen Li, Zhuo Jin, Linyi Qian, and Hailiang Yang. 2025. Textual Analysis of Insurance Claims with Large Language Models. *Journal of Risk and Insurance* 92, 2 (2025), 505–535. https://doi.org/10.1111/jori.70004

[38] Shuyue Stella Li, Vidhisha Balachandran, Shangbin Feng, Jonathan S. Ilgen, Emma Pierson, Pang Wei Koh, and Yulia Tsvetkov. 2024. MediQ: Question-Asking LLMs and a Benchmark for Reliable Interactive Clinical Reasoning. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Neural Information Processing Systems Foundation, Inc., 28858–28888. https://doi.org/10.52202/079017-0908

[39] Zekun Li, Shinda Huang, Jiangtian Wang, Nathan Zhang, Antonis Antoniades, Wenyue Hua, Kaijie Zhu, Sirui Zeng, Chi Wang, William Yang Wang, and Xifeng Yan. 2026. SOPBench: Evaluating Language Agents at Following Standard Operating Procedures and Constraints. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, Hung-Yi Lee, Sunipa Dev, Junyi Jessy Li, and Desmond Elliott (Eds.). Association for Computational

Linguistics, Budapest, Hungary, 21561–21616. https://doi.org/10.18653/v1/2026.emnlp-main.936

[40] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: A Programming Language for the Law. *Proceedings of the ACM on Programming Languages* 5, ICFP (2021), 1–29. https://doi.org/10.1145/3473582

[41] James Mohun and Alex Roberts. 2020. *Cracking the Code: Rulemaking for Humans and Machines*. OECD Working Papers on Public Governance 42. OECD Publishing. https://doi.org/10.1787/3afe6ba5-en

[42] Object Management Group. 2016. Case Management Model and Notation (CMMN), Version 1.1. OMG formal specification, OMG Document Number formal/2016-12-01. https://www.omg.org/spec/CMMN/1.1

[43] Object Management Group. 2024. Decision Model and Notation (DMN), Version 1.5. OMG formal specification, OMG Document Number formal/24-01-01. https://www.omg.org/spec/DMN/1.5

[44] Theo Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua Tenenbaum, and Roger Levy. 2023. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 5153–5176. https://doi.org/10.18653/v1/2023.emnlp-main.313

[45] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 3806–3824. https://doi.org/10.18653/v1/2023.findings-emnlp.248

[46] Raja Parasuraman and Dietrich H. Manzey. 2010. Complacency and Bias in Human Use of Automation: An Attentional Integration. *Human Factors: The Journal of the Human Factors and Ergonomics Society* 52, 3 (2010), 381–410. https://doi.org/10.1177/0018720810376055

[47] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. 2007. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems* 24, 3 (2007), 45–77. https://doi.org/10.2753/MIS0742-1222240302

[48] M. Pesic and W. M. P. van der Aalst. 2006. A Declarative Approach for Flexible Business Processes Management. In *Business Process Management Workshops (BPM 2006 International Workshops, Proceedings) (Lecture Notes in Computer Science)*, Johann Eder and Schahram Dustdar (Eds.). Springer, Berlin, Heidelberg, 169–180. https://doi.org/10.1007/11837862_18

[49] Raymond Reiter. 1978. On Closed World Data Bases. In *Logic and Data Bases*, Hervé Gallaire and Jack Minker (Eds.). 55–76. https://doi.org/10.1007/978-1-4684-3384-5_3

[50] R. Reiter. 1980. A Logic for Default Reasoning. *Artificial Intelligence* 13, 1–2 (1980), 81–132. https://doi.org/10.1016/0004-3702(80)90014-4

[51] Marzieh Saeidi, Max Bartolo, Patrick Lewis, Sameer Singh, Tim Rocktäschel, Mike Sheldon, Guillaume Bouchard, and Sebastian Riedel. 2018. Interpretation of Natural Language Rules in Conversational Machine Reading. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2087–2097. https://doi.org/10.18653/v1/D18-1233

[52] Max Schemmer, Niklas Kühl, Carina Benz, Andrea Bartos, and Gerhard Satzger. 2023. Appropriate Reliance on AI Advice: Conceptualization and the Effect of Explanations. In *Proceedings of IUI 2023*. 410–422. https://doi.org/10.1145/3581641.3584066

[53] Maung K. Sein, Ola Henfridsson, Sandeep Purao, Matti Rossi, and Rikard Lindgren. 2011. Action Design Research. *MIS Quarterly* 35, 1 (2011), 37–56. https://doi.org/10.2307/23043488

[54] M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. 1986. The British Nationality Act as a Logic Program. *Commun. ACM* 29, 5 (1986), 370–386. https://doi.org/10.1145/5689.5920

[55] Sergio Servantez, Joe Barrow, Kristian Hammond, and Rajiv Jain. 2024. Chain of Logic: Rule-Based Reasoning with Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2721–2733. https://doi.org/10.18653/v1/2024.findings-acl.159

[56] Bas C. van Fraassen. 1966. Singular Terms, Truth-Value Gaps, and Free Logic. *The Journal of Philosophy* 63, 17 (1966), 481–495. https://doi.org/10.2307/2024549

[57] Maxim Vidgof, Stefan Bachhofner, and Jan Mendling. 2023. Large Language Models for Business Process Management: Opportunities and Challenges. In *Business Process Management Forum (BPM 2023) (Lecture Notes in Business Information Processing)*, Chiara Di Francescomarino, Andrea Burattin, Christian Janiesch, and Shazia Sadiq (Eds.). Springer, 107–123. https://doi.org/10.1007/978-3-031-41623-1_7

[58] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=1PL1NIMMrw

[59] Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. 2025. Know Your Limits: A Survey of Abstention in Large Language Models. *Transactions of the Association for Computational Linguistics* 13 (2025), 529–556. https://doi.org/10.1162/tacl_a_00754

[60] Ruixuan Xiao, Wentao Ma, Ke Wang, Yuchuan Wu, Junbo Zhao, Haobo Wang, Fei Huang, and Yongbin Li. 2024. FlowBench: Revisiting and Benchmarking Workflow-Guided Planning for LLM-based Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 10883–10900. https://doi.org/10.18653/v1/2024.findings-emnlp.638

[61] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR 2025)*. https://openreview.net/forum?id=roNSXZpUDN

[62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=WE_vluYUL-X

[63] Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. SatLM: Satisfiability-Aided Language Models Using Declarative Prompting. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Neural Information Processing Systems Foundation, Inc., 45548–45580. https://doi.org/10.52202/075280-1974

[64] Xuan Zhang, Yang Deng, Zifeng Ren, See-Kiong Ng, and Tat-Seng Chua. 2024. Ask-before-Plan: Proactive Language Agents for Real-World Planning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 10836–10863. https://doi.org/10.18653/v1/2024.findings-emnlp.636

A PROOFS

Proposition 3.1. Strong Kleene connectives are monotone in the information order in which U lies below both T and F [33]. Equations (1) and (2) compose such connectives over the acyclic graph, so by induction in topological order every value in {T, F} persists in every completion. The proof needs the two scope conditions of the statement. The equations must be the workflow's semantics, which holds for flow workflows: outside them, a relation that suppresses or blocks a step can turn an active step inactive when a fact becomes known, and the proposition can fail, for instance when a suppressing transition under an open fact enters a step that is also blocked. Completions must range over {T, F} for every fact independently.

The tested workflows are flow workflows. We checked this for the four workflows of Study 3 with the product's own code at commit 6a72159c (`process/workflows.py`). The product walks one path: at each step it follows the first transition whose condition holds. Equation (1) activates every step reached through a transition that holds. For each tenancy workflow we enumerated every complete assignment of the facts its conditions use (32, 32 and 8) and compared the documents the product's walk lists with those of Equations (1) and (2); all agreed. In one step of the defect workflow two transitions can hold together, and both lead to the same later step, so the two semantics activate the same steps. For every three-valued assignment (243, 243 and 27) the product's evaluation also requested and held exactly what Equation (3) gives. Optional documents were left out of both comparisons: the product lists them but never requests or holds them. The theft workflow is one step without transitions, so Equation (1) activates only that step and Equation (2) reduces to each document's own condition. `analysis/check_fragment.py` repeats these comparisons. Before the run, the driver's preflight check compared the product's evaluator with the reference engines (Supplement, Section S10).

The plain walk on theft. Above eight open facts, or 256 combinations of answers, the product returns the plain walk, which evaluates each condition in strong Kleene logic and stops at the first transition whose condition is open. Every theft case-run of Study 3 was evaluated this way (384 of 384 for CasePath). On the theft workflow this changes nothing. It has no transitions, and each of its document conditions is empty, one fact tested for true, or a disjunction of distinct facts each

tested for true. Strong Kleene evaluation of such a formula is T if a disjunct is T, F if all are F, and U otherwise; U is exactly the case in which two completions disagree. On 5,000 random three-valued assignments the product's evaluation and Equation (3) agreed on every document (`analysis/check_fragment.py`). This check differs from the preflight check described in the Supplement, Section S10, which compared the product's evaluator with the reference engine on 3,000 random theft assignments.

Proposition 3.2. In the tenancy defect workflow, a flow workflow over two-valued facts, the intake step branches to a safety step when health effects are alleged and directly to the timeline step otherwise; the safety step leads to the timeline step as well. With $v(\mathit{health}) = \mathsf{U}$, Equation (1) gives $a(\mathit{timeline}) = (\mathsf{U} \wedge \mathsf{T}) \vee (\mathsf{T} \wedge \mathsf{U}) = \mathsf{U}$, so every document of the timeline step and of the later steps it reaches is held, unless its own or its branch condition is F. Yet under both completions $a(\mathit{timeline}) = \mathsf{T}$.

Proposition 3.3. (a) By Equation (3), $\hat{r}(d) = \mathsf{U}$ exactly when two completions disagree on $r(d)$; this uses only that the need of d is defined under each complete assignment. Soundness holds by definition: a value in $\{\mathsf{T}, \mathsf{F}\}$ is the value under every completion. (b) If $r(d) \neq \mathsf{U}$, Proposition 3.1, which needs a flow workflow, gives the same value under every completion, so $\hat{r}(d) = r(d)$.

Proposition 3.4. (a) Under the assumption, v^* is a completion of v. If $\hat{r}_v(d) = \mathsf{T}$, every completion needs d, so $r_{v^*}(d) = \mathsf{T}$ and the request is needed; if $\hat{r}_v(d) = \mathsf{F}$, then $r_{v^*}(d) = \mathsf{F}$ and marking d not needed is right. The remaining documents are held, which is not an unsafe error. Whether a needed document is on file is taken from the document judgments; a wrong judgment there is outside the proposition. (b) The checklist is computed from v alone, so a wrong value enters the evaluation exactly as a correct one would. A document with $\hat{r}_v(d) = \mathsf{T}$ and $r_{v^*}(d) = \mathsf{F}$ is an unnecessary request; one with $\hat{r}_v(d) = \mathsf{F}$ and $r_{v^*}(d) = \mathsf{T}$ is missed.

Exact computation of reading errors. To quantify both kinds without depending on any model, we computed exactly, for all 150 tenancy claims of Study 1 and every partial assignment of their facts, which documents each semantics requests and holds (`scicore/s1_propagation.py`). The noise model judges each fact of a claim correctly with probability $1 - \varepsilon$ and misjudges it otherwise, independently of the other facts; a misjudgement is unresolved with probability ρ and the opposite value with probability $1 - \rho$. We report $\varepsilon = 0.1$ with $\rho = 1$ (every misjudgement unresolved) and $\rho = 0$ (every misjudgement a wrong value). The expectations are exact sums over all assignments, without sampling. Counts are expected numbers of documents summed over the claims; unnecessary requests are counted beyond those made with no misjudgement. Of the 1002 documents that the claims need, unresolved values become 84.5 expected holds under exact deferral and no unsafe error. Strong Kleene evaluation holds 68% more; reading unresolved values as false misses 84.5 documents, and reading them as true adds 39.5 unnecessary requests. Wrong values cause 84.5 missed documents and 37.0 unnecessary requests under every semantics. The value 84.5 appears three times for one reason: for every pattern of misjudged facts in these claims, the required documents exact deferral holds when the facts are unresolved are exactly those lost when the facts are read as false, and exactly those lost when they take the opposite value. Each pattern has the same probability in the three computations, so the three expectations are equal. Figure 7 shows the same quantities per 100 required documents.

B PROTOCOL OF THE DEPLOYMENT STUDY

This protocol is version 1.0-rc, to be frozen on 20 November 2026 as version 1.0. No part of the study has run, and this article reports no result of it. The full text, with every record field,



Fig. 7. What reading errors become, computed exactly over all 150 tenancy claims of Study 1, with each fact misjudged with probability 0.1: above, every misjudgement unresolved; below, every misjudgement the opposite value. Bars give expected documents per 100 of the 1002 required documents, that is, percent. Under strong Kleene evaluation the light part of the bar is the holds beyond those of exact deferral. Unnecessary requests are counted beyond those made with no misjudgement. Three-valued evaluation turns unresolved values into holds, two-valued evaluation turns them into missed or unnecessary requests, and a wrong value becomes an error under every semantics.

is `research/tmis2026/expert_study/PREREGISTRATION_DRAFT.md` in the repository [36]; the compact version written before the change of 12 November 2026 is in the Supplement, Section S16.

Questions. The primary endpoint (Q1) asks how far CasePath agrees with what the insurer did next, at every decision point of four sealed files, compared with a single prompt; it needs no expert time. Secondary endpoints ask what five of the insurer's claims experts do with the same items (Q2), what work with CasePath looks like (Q3), whether CasePath improves from corrections on a case whose corrections it has not seen (Q4), and why experts change things (Q5).

Cases and decision points. Eleven closed tenancy-defect (mould) files of the insurer: four sealed (D2, D4, D5, D7) and seven for development. The handler workflow was written by the product's builders from the seven development files and the law. A *decision point* is the state of a file immediately before a recorded outgoing action of the insurer, such as a letter or e-mail; every such action makes one. For each, a *cut copy* holds only the documents that precede the action. All runs at a decision point use its cut copy, and nobody in a session sees anything dated after it.

Where the reference comes from. There is no gold label. *Practice* (P) is what the insurer did in the action: the documents it requested, the kind of action and the follow-up period. Two transcriptions map P to a frozen catalogue: T1 by the product builder's automated labeller, and T2 by a session that did not build the product and sees neither T1 nor any output. The study lead, an author and one of the product's builders, not a claims expert, settles their disagreements against the file before any run on a sealed cut copy. P is practice, not proof of correctness.

Measures of the primary endpoint. Per decision point, for CasePath at the frozen commit and for a single-prompt comparator given the same handler workflow and catalogues: (a) routing to the handler workflow; (b) of the documents in P, the number CasePath requests now, its further requests not in P, and the share of its requests that are in P; (c) the same kind of next action as the insurer's; (d) the follow-up period against the insurer's; (e) descriptively, how many deciding facts

were stated, open or not addressed, and how the system treated each. Counts carry exact binomial intervals, per file and pooled. No hypothesis test is run on this endpoint: with four files as clusters, no test has useful power.

Prediction rule. At the freeze, the builders' counts on the seven development files are recorded. For measures (a) to (d), the exact two-sided 80% interval of each development proportion is applied to the sealed denominator. A pooled sealed count below its lower bound is reported as "below the development result", one above its upper bound as "above", and the rest as "consistent". The protocol predicts lower agreement on the sealed files than in development, because the development counts are in sample and come mostly from the first request of each file.

Secondary endpoint. Each expert works through three sealed files in the product, cut before the insurer's first request for documents, and leaves, changes or annotates each item. The only pre-stated test is a one-sided sign test over the five experts, whose smallest attainable p is 1/32.

Order after the freeze. (1) The product, the handler workflow and its catalogue are frozen and hashed. (2) The decision points are found, and the cut copies made and checked for leaks. (3) T1 and T2 transcribe P, each without the other's record or any output. (4) The study lead settles their disagreements; P is hashed. (5) The runs are made on the cut copies. (6) The archival results are computed and an archival hash is recorded. (7) No expert session starts before that hash exists. Nobody who can change the product reads a sealed file after its first decision point before the product's freeze hash exists.

Stated limits. Four files of one insurer and one process family bound every inference. The sealed files were among the files the product was developed on. The handler workflow was written by the product's builders, not by the insurer's process owner, and may copy the habits of seven files. The product's seed Knowledge was built from the insurer's request letters on the development files, so agreement with practice is partly agreement with what the product learned from it. P rests on one automated labeller and one independent transcription, settled by an author. Decision points within a file are not independent, so pooled intervals are too narrow. The interface the experts use is a later version than the tested commit's. An expert may have handled a study file before. Only counts, statuses and relative times leave the insurer; files and texts stay with it.

C DEVIATIONS AND IRREGULARITIES OF STUDY 3

Each item below is stated in one line; the Supplement, Section S10, gives every item in full. The plan (`ANALYSIS_PLAN_STUDY3.md`) was committed before any design ran on `fresh2/`, and the analysis was run once on the stored outputs. Studies 1 and 2 are covered in the Supplement, Section S3.

The run.

- The run needed 5 invocations of the driver; 4 stopped early, each resumed from the response cache with product, model, data and analysis unchanged.
- The first was interrupted by the operator to add parallel workers; its log line ("interrupted during the checks", with a wrong spend) is wrong and is corrected by a dated note.
- The second was killed by the session's time limit on background commands; the wrapper's interrupt had no effect.
- The third ended when the container was restarted; disk, cache and worktree were unchanged.
- The fourth was stopped, as planned, by the wrapper before the time limit; the fifth completed the run.
- Workers rose from 6 to 20 after the first invocation, and the main spending cap was lowered at each resumption to what remained of USD 32; the plan says settings stay unchanged.

- The ledgers record USD 34.33 for the run (22.41 cost-efficient, 11.92 frontier model).
- 50 comparator outputs failed and were scored as empty checklists, as planned: 40 from ReAct, none from a product design; a post-hoc analysis leaves ReAct's failures out.
- No model snapshot, temperature or seed is recorded; the provider's defaults applied.

Before the run.

- The run started before the product's owners confirmed the merge of the tested commit; at the time of writing they have not confirmed it.
- The test texts were written by an AI agent session with read access to the product's prompts and development results (Appendix D).
- The probe claims were written knowing the test set's design but not its texts; the plan's "writer that had not seen `fresh2/`" holds for the texts only.
- The hypotheses were reordered 38 minutes before the freeze, on development evidence (Section 6.2).
- The open rule overruled a criterion on wrong holds that the directing author had set earlier.
- Two comparator texts changed during Study 2's development; Model executes rules and ReAct were written during the product's development and run on development data before the freeze.

Departures from the plan's text.

- Calls per case differ from the plan's design table (CasePath about 26 planned, 28.1 and 30.1 measured).
- Self-consistency costs less than CasePath, checklist only, on theft, so no design is matched in cost as the plan assumed.
- The full product and checklist only share cached responses, so their contrast is not between independent runs.
- The Core's runs share readings and are not independent.
- Cost per case joins each call to the ledger by its cache key; some agent costs could not be attributed.
- The subsets were selected from the reference; their counts equal the plan's.
- Subset rows beside H2 and H3 are family-mean contrasts, marked descriptive.
- 2 secondary contrasts are not listed in the plan.
- H5's non-inferiority p is reported outside the procedure.
- The plan fixes the number of resamples; the specification fixes the seed.
- Two-valued with unresolved taken as false equals CasePath on theft, because theft scores requests only.

After the results.

- Thirteen analyses were added after a first referee report and claims audit, and further checks after an outside review; none is registered or changes a status (Supplement, Sections S12.2 and S13).
- From the end of the test, the theft stories of `fresh2/` are development data; the diagnosis and the lint used them.
- The lint's mutation test made the only new model calls (USD 0.15).
- One clean-room evaluator listed a scratch directory after delivering its hashed reference and saw other file names; by its report it opened none.
- Changes for speed live on a separate branch, not the tested commit.

D REPRODUCIBILITY AND USE OF GENERATIVE AI

Release. The repository [36] holds the tested product at commit 6a72159c, whose parity command prints the source-tree hash `03048020. . .` ; the research workflows; the synthetic data of all three studies; the harness, the analysis scripts and every stored checklist, ledger and run log; and the frozen plans. The Study 3 response cache (about 23,000 files) is offered as a separate archive. At the tested commit the source names the insurer's subsidiary as the default organisation, so the public release is either a scrubbed commit with its own parity hash or the tested commit with the insurer's approval. The insurer's workflows, Knowledge and files are not released. Supplement, Section S15 gives the details and the licences.

Reproducing without model calls. One command, `reproduce.sh`, re-checks the stored results from a clone: data manifests, the parity hash, the claims of Appendix A, Study 3's registered analysis and the analyses added after the first review, with their number macros, byte for byte. The checks added after the outside review (Sections 6.3 to 6.7) are not yet a step of that command; `analysis/verify_paper.py` regenerates their macros from the stored result files. Neither re-checks the model runs, whose stored outputs are their inputs. A re-run with model calls needs the reader's own model endpoint, and its answers may differ.

Generative AI. The two evaluated models are the objects of study. An AI agent session wrote the test texts, and sessions of another model of the same family checked them blind. AI editor sessions reviewed the theft rules and the lint's flags; AI sessions wrote the clean-room references. No jurist and no person checked these sessions' work. AI coding and writing assistants helped implement the harness and the product changes, run the analyses and draft the text, under the authors' direction; the authors are responsible for all of it.